



UNIVERSITÀ DEGLI STUDI DI TRENTO

Dipartimento di Ingegneria e Scienza dell'Informazione

Appunti di Basi di dati

Lezioni, e Simulazioni d'esame

A cura di:

Mirco Negri

Versione Light Mode. Disponibile anche la [versione Dark Mode](#).

ottobre 2026

Indice

1	Le Basi di Dati e i loro Utenti	5
1.1	Cos'è una Base di Dati	5
1.2	DBMS e Database System	5
1.3	Architettura di un Database System	5
1.4	Cosa Deve Fornire un DBMS	6
1.5	Interazione tra Applicazioni e Basi di Dati	6
1.6	Le Origini del Modello Relazionale	6
1.7	Indipendenza dei Dati	6
1.7.1	Indipendenza Logica dei Dati	7
1.7.2	Indipendenza Fisica dei Dati	7
1.8	Livelli di Astrazione dei Dati	7
1.9	Perché Usare una Base di Dati	8
1.9.1	Condivisione dei Dati e Transazioni Multi-utente	8
1.10	Digressione: OLTP vs. OLAP	8
1.11	Utenti di una Base di Dati	9
1.11.1	Tipi di Utilizzatori	9
1.12	Quando è Meglio NON Usare un DBMS	9
2	Il Modello Relazionale	10
2.1	Concetti Generali	10
2.2	Cos'è una Relazione	10
2.2.1	Definizione Intensionale	10
2.2.2	Definizione Estensionale	11
2.3	Stato di una Relazione	11
2.4	Chiave di una Relazione	11
2.5	Terminologia	11
2.6	Vincoli	11
2.7	Vincolo di Chiave	12
2.8	Vincolo di Integrità dell'Entità (Entity Integrity)	12
2.9	Vincoli di Integrità Referenziale	13
2.10	Schema di un Database Relazionale	14
2.11	Altri Tipi di Vincolo	14
2.12	Stato di una Base di Dati Relazionale	14
2.13	Operazioni di Aggiornamento e Violazione dei Vincoli	14
2.13.1	Violazioni per Operazione INSERT	15
2.13.2	Violazioni per Operazione DELETE	15
2.13.3	Violazioni per Operazione UPDATE	15
2.14	Come Preservare l'Integrità Referenziale	15
3	Algebra Relazionale	16
3.1	Introduzione	16
3.2	Schema e Dati di Riferimento	17
3.3	Selezione (SELECT, σ)	17
3.4	Proiezione (PROJECT, π)	18
3.5	Unione, Differenza e Intersezione	18
3.6	Prodotto Cartesiano ($\times$)	18
3.7	Ridenominazione (RENAME, ρ)	18

3.8	Join	18
3.8.1	Theta-Join	18
3.8.2	Join Naturale	19
3.8.3	Equijoin	19
3.9	Divisione ($\div$)	19
3.10	Outer Join	19
3.11	Espressioni ed Alberi di Query	20
4	SQL	20
4.1	Cos'è SQL	20
4.2	Istanza di Riferimento	20
4.3	Struttura Base di una Query	20
4.3.1	Semantica Valutativa (concettuale)	21
4.4	Operatori di Confronto	21
4.5	Operatori di Aggregazione	22
4.6	GROUP BY e HAVING	22
4.6.1	Esempio (Q32)	23
4.7	Interrogazioni Annidate (Nested Queries)	23
4.7.1	Query Indipendenti e Correlate	23
4.7.2	Casi d'Uso	23
4.7.3	Annidamento a Più Livelli e Operatori ANY/ALL	24
4.8	Operatori Insiemistici	24
4.9	JOIN	24
4.9.1	INNER JOIN vs. Prodotto Cartesiano + WHERE	24
4.9.2	Esempio di LEFT JOIN	25
4.9.3	ANTI JOIN	25
4.10	Linee Guida Pratiche per Scrivere Query SQL	25
5	Progettazione Concettuale: il Modello Entity-Relationship (ER)	26
5.1	La Progettazione di una Base di Dati	26
5.2	Concetti Fondamentali: Entità e Attributi	27
5.2.1	Tipi di Attributo	27
5.2.2	Attributi Chiave	27
5.2.3	Rappresentazione Grafica	27
5.3	Relazioni (Relationships)	27
5.3.1	Grado di una Relazione	27
5.3.2	Attributi della Relazione	28
5.3.3	Relationship Type vs. Relationship Set	28
5.4	Vincoli sulle Relazioni	28
5.4.1	Vincolo di Chiave (key constraint)	28
5.4.2	Vincoli di Cardinalità (cardinality ratio)	28
5.4.3	Vincolo di Partecipazione (participation constraint)	28
5.4.4	Notazione (min, max)	29
5.5	Relazioni Ricorsive	29
5.6	Entity Types Deboli	29
5.7	Gerarchie di Classi (ISA)	29
5.7.1	Vincoli: Overlap e Coverage	30
5.8	Aggregazione	30
5.9	Dai Requisiti al Modello: Regole Empiriche	30

5.10	Scelte di Progettazione Concettuale con ER	31
5.10.1	Entità vs. Attributo	31
5.10.2	Entità vs. Relazione	31
5.10.3	Relazioni Binarie o Ternarie	31
5.10.4	Aggregazione o Relazione Ternaria?	32
6	Mapping dal Modello ER al Modello Relazionale	32
6.1	Obiettivi del Mapping	32
6.2	Da Entity Type a Tabelle	32
6.3	Mapping delle Relazioni (Relationship Types)	33
6.3.1	Caso Base (senza vincoli)	33
6.3.2	Relazioni Ricorsive	33
6.3.3	Vincolo di Chiave	33
6.3.4	Vincolo di Partecipazione	34
6.4	Weak Entities	35
6.5	Gerarchie IS-A	36
6.5.1	Metodo Generale	36
6.5.2	Metodo di Riduzione	36
6.5.3	Confronto tra i Due Metodi	37
6.6	Aggregazioni	37
7	Dipendenze Funzionali e Chiavi	37
7.1	Motivazione: Anomalie nella Base di Dati	37
7.2	Definizione Formale di Dipendenza Funzionale	38
7.3	Come si Identificano le DF	38
7.4	DF Banali	38
7.5	DF e Chiavi	38
7.6	DF e Anomalie	39
7.7	Chiusura di un Insieme di Dipendenze Funzionali	39
7.7.1	Assiomi di Armstrong	39
7.8	Chiusura di un Insieme di Attributi	39
7.9	Individuazione delle Superchiavi	40
7.9.1	Algoritmo per le Chiavi Candidate	40
7.10	Copertura Minimale	41
8	Normalizzazione	42
8.1	Normalizzazione via Decomposizione	42
8.2	Proprietà Desiderabili di una Decomposizione	42
8.2.1	Perdita di Informazione: Esempio	42
8.2.2	Condizione Formale per la Decomposizione Senza Perdita	42
8.2.3	Conservazione delle DF	42
8.3	Prima Forma Normale (1NF)	43
8.4	Richiami: Attributi Primi e Non-Primi	43
8.5	Seconda Forma Normale (2NF)	43
8.6	Terza Forma Normale (3NF)	44
8.7	3NF e Ridondanza Residua	44
8.8	Forma Normale di Boyce-Codd (BCNF)	44
8.8.1	Procedura di Decomposizione in BCNF	44
8.8.2	Quando la BCNF Non È Raggiungibile Preservando le DF	44

8.9	Verificare una DF su un'Istanza	45
8.10	Esempio Completo: dalle Chiavi alla BCNF	46

1 Le Basi di Dati e i loro Utenti

1.1 Cos'è una Base di Dati

Una **base di dati** (BdD) è una raccolta organizzata (strutturata) di informazioni che devono essere gestite per un periodo di tempo lungo, anche di molti anni. Più precisamente, una base di dati è una raccolta di informazioni gestita da un **database management system** (DBMS).

Esempi di Basi di Dati:

- Dati relativi a studenti, corsi ed esami di un'università
- Il catalogo dei prodotti di un marketplace elettronico
- I *Points of Interest* (POI) su una mappa (es. Google Maps)
- Il Pubblico Registro Automobilistico (PRA) dell'Automobile Club Italiano (ACI)
- I pagamenti delle tasse dei contribuenti di un Comune
- Gli orari dei voli e le prenotazioni dei passeggeri di una compagnia aerea

1.2 DBMS e Database System

- **Database Management System (DBMS)**: pacchetto software progettato per gestire basi di dati informatizzate.
- **Database System**: l'insieme costituito dal software del DBMS insieme ai dati memorizzati nella base di dati. Talvolta nella definizione sono incluse anche le applicazioni software esterne che interagiscono con il DBMS per eseguire i propri compiti.

1.3 Architettura di un Database System

- Gli **utenti non esperti** (*unsophisticated*: clienti, agenti di viaggio, ...) interagiscono tramite *Web Forms* o *Application Front Ends*; gli **utenti esperti** (*sophisticated*: programmatori, amministratori) interagiscono tramite un'*SQL Interface*. Entrambi generano comandi SQL.
- Il **Query Evaluation Engine** riceve i comandi ed è composto da *Parser*, *Optimizer*, *Plan Executor* e *Operator Evaluator*.
- Il livello sottostante gestisce *Files and Access Methods*, il *Buffer Manager* e il *Recovery Manager*, coordinati dal blocco di **Concurrency Control** (*Transaction Manager* e *Lock Manager*).
- Alla base, il *Disk Space Manager* gestisce fisicamente gli **Index Files**, i **Data Files** e il **System Catalog** che compongono il database.

1.4 Cosa Deve Fornire un DBMS

Un DBMS deve:

- supportare la memorizzazione di grandi quantità di dati;
- gestire la robustezza dei dati e il loro recupero in caso di fallimenti, errori o azioni malevole;
- controllare l'accesso ai dati di più utenti contemporaneamente (**concorrenza**), evitando interazioni indesiderate tra utenti (*isolation*) e modifiche incomplete dei dati (*atomicity*).

Per la gestione dei dati sono previsti diversi linguaggi specializzati:

- **DDL** (*Data Definition Language*): definisce gli schemi per strutturare i dati;
- **DML** (*Data Manipulation Language*, o *query language*): modifica i dati e li estrae dalla base di dati;
- **DCL** (*Data Control Language*): gestisce l'accesso ai dati (**grant**, **revoke**, ...);
- **TCL** (*Transaction Control Language*): gestisce la sicurezza delle transazioni (**commit**, **rollback**, ...).

1.5 Interazione tra Applicazioni e Basi di Dati

I programmi applicativi interagiscono con la base di dati generando:

- **Interrogazioni** (*queries*): accesso a differenti porzioni di dati e formulazione della risposta;
- **Transazioni** (*transactions*): sequenze atomiche di azioni (letture/scritture) sulla base di dati.

Le applicazioni non devono consentire l'accesso ai dati ad utenti non autorizzati.

1.6 Le Origini del Modello Relazionale

Un enorme impulso allo sviluppo delle basi di dati è venuto dal lavoro di **Ted Codd** (1970), che ha introdotto l'idea di mostrare agli utenti una **visione logica dei dati**, indipendente dalla struttura fisica. Le interrogazioni vengono così espresse attraverso un linguaggio di alto livello, che utilizza il modello logico e non quello fisico dei dati.

1.7 Indipendenza dei Dati

La separazione tra programmi e dati è spesso chiamata **indipendenza programmi-dati**: permette di modificare le strutture dati e l'organizzazione fisica dei dati senza dover cambiare l'accesso dei programmi alla base di dati via DBMS.

Nei sistemi basati su file, invece, la struttura dei file di dati è contenuta nei programmi che devono accedervi: se cambia la struttura del file, è probabile che debbano essere modificati anche i programmi che vi accedono, rendendo difficile e costosa la manutenzione del software nel tempo.

1.7.1 Indipendenza Logica dei Dati

- **Definizione:** capacità di modificare lo schema logico del database senza dover alterare lo schema esterno o le applicazioni che lo utilizzano.
- **Vantaggi:** consente di aggiungere, rimuovere o modificare campi e tabelle senza influire sulle applicazioni esistenti; migliora la flessibilità e la gestione dell'evoluzione del database.
- **Esempio:** aggiungere un nuovo attributo a una tabella senza modificare il codice delle applicazioni che vi accedono.

1.7.2 Indipendenza Fisica dei Dati

- **Definizione:** capacità di modificare lo schema fisico del database senza influenzare lo schema logico o le applicazioni.
- **Vantaggi:** permette di ottimizzare le prestazioni (es. cambiando la struttura di memorizzazione o gli indici) senza alterare la logica delle applicazioni; migliora la scalabilità e la gestione delle risorse hardware.
- **Esempio:** spostare un database su un nuovo hardware, o ristrutturarne l'archiviazione, senza cambiare la modalità di accesso ai dati per l'utente finale.

1.8 Livelli di Astrazione dei Dati

- **Schema logico:** modello che rappresenta i dati nei termini del modello proprio del DBMS (es. il modello relazionale nei RDBMS);
- **Schema fisico:** dettagli su come le relazioni descritte dallo schema logico sono memorizzate su memoria secondaria (es. file di record ordinati/non ordinati, tipo di indici creati, ecc.);
- **Schema esterno:** una o più **viste** specifiche per velocizzare/facilitare/autorizzare l'accesso a insiemi di dati a specifici utenti o gruppi di utenti.

I tre livelli seguono il flusso: *livello fisico* $\rightarrow$ *livello logico* $\rightarrow$ *livello esterno (viste)*.

Esempio. Schema logico per un DB universitario:

```
Students(sid: string, name: string, login: string, age: integer, gpa: real)
Faculty(fid: string, fname: string, sal: real)
Courses(cid: string, cname: string, credits: integer)
Rooms(rno: integer, address: string, capacity: integer)
Enrolled(sid: string, cid: string, grade: string)
Teaches(fid: string, cid: string)
Meets_In(cid: string, rno: integer, time: string)
```

- **Schema fisico:** file non ordinato di record, con un indice B+Tree sulla prima colonna di STUDENTS e COURSES.
- **Schema esterno:** una relazione che mostri tutti i corsi con relativo docente e numero di iscritti: `Courseinfo(cid: string, fname: string, enrollment: integer)`.

1.9 Perché Usare una Base di Dati

- Indipendenza logica e fisica dei dati (vista sopra);
- Accesso concorrente ai dati;
- Crash recovery;
- Integrità dei dati e sicurezza;
- Amministrazione dei dati;
- Riduzione dei tempi di sviluppo di applicazioni.

1.9.1 Condivisione dei Dati e Transazioni Multi-utente

La condivisione dei dati permette a gruppi di utenti di leggere e scrivere contemporaneamente (*concurrent users*) sulla stessa base di dati:

- il **controllo della concorrenza** nel DBMS garantisce che ogni transazione sia eseguita correttamente o annullata;
- il **sottosistema di recovery** assicura che ogni transazione correttamente completata sia memorizzata in modo permanente.

Questo permette di eseguire centinaia di transazioni concorrenti al secondo mantenendo validità e coerenza dei dati: si parla di **OLTP** (*Online Transaction Processing*).

1.10 Digressione: OLTP vs. OLAP

	OLTP	OLAP
Finalità	Gestire le transazioni quotidiane di un'azienda (es. ordini, pagamenti)	Supportare l'analisi complessa dei dati per BI e reporting
Operazioni	Alta frequenza, operazioni brevi e semplici (insert , update , delete)	Bassa frequenza, query complesse e di lunga durata
Priorità	Integrità/coerenza dei dati; risposta rapida alle query	Query ad-hoc, analisi multidimensionale
Schema	Normalizzato, per minimizzare la ridondanza	Denormalizzato (es. <i>star schema</i>), per le performance
Esempi	E-commerce, banche, punti vendita	Reporting aziendale, dashboard di BI

Tabella 1: Confronto tra sistemi OLTP e OLAP

Sistemi per OLTP:

- **Database Relazionali (RDBMS)**: MySQL, MariaDB, PostgreSQL, Microsoft SQL Server, IBM DB2, Oracle Database;

- **Software ERP:** SAP, Oracle, Panthera, TeamSystem, ...

Sistemi per OLAP:

- **Data Warehousing Platforms:** Amazon Redshift, Google BigQuery, Snowflake, Microsoft Azure Synapse Analytics;
- **Business Intelligence Tools:** Tableau, Power BI, Qlik Sense, SAP BusinessObjects.

1.11 Utenti di una Base di Dati

Gli utenti si dividono in due gruppi principali:

- chi utilizza e controlla il contenuto della base di dati, insieme a chi progetta e sviluppa applicazioni su di essa (**utilizzatori**);
- chi progetta e sviluppa il software del DBMS con tutti i suoi sottosistemi (**sviluppatori**).

1.11.1 Tipi di Utilizzatori

- **Database Administrator** (Amministratore): responsabile dell'autorizzazione di accesso alla base di dati, del coordinamento e monitoraggio del suo utilizzo, dell'acquisto di risorse software/hardware, del monitoraggio dell'efficienza operativa del sistema.
- **Database Designer** (Progettista): responsabile della definizione del contenuto, dello schema, dei vincoli e delle transazioni eseguibili sulla BD; comunica con gli utenti per raccoglierne le necessità.
- **End-users** (Utenti finali): utilizzano i dati mediante query e report; alcuni possono aggiornare i contenuti. Si suddividono in:
 - *Casual* (Occasionali): accedono alla base di dati solo occasionalmente, quando necessario (es. top manager che leggono dati fortemente aggregati);
 - *Naïve o Parametric* (non-esperti): la maggioranza degli utenti finali; usano funzioni predefinite (le *canned transactions*) per interagire con la BD (es. cassieri di banca, impiegati di segreteria, utenti di social media o applicazioni web).

1.12 Quando è Meglio NON Usare un DBMS

Cosa può inibirne l'adozione:

- alti investimenti iniziali e necessità di hardware dedicato;
- costi aggiuntivi per sicurezza, controllo di concorrenza, recovery, funzioni di integrità.

Quando un DBMS potrebbe non essere necessario:

- se la base di dati e l'applicazione sono molto semplici, ben definite e non soggette a modifiche continue;
- se non è richiesto l'accesso di più utenti.

Quando adottarlo può essere impossibile o sconsigliato:

- in molti *embedded systems*, dove mancano adeguate capacità di memorizzazione per un DBMS *general purpose*;
- con requisiti di tempo reale molto stringenti (es. sistemi di switch telefonico);
- quando il modello dati del DBMS non è abbastanza espressivo per la complessità dei dati (es. basi di dati genomiche/proteiche);
- se servono operazioni speciali non supportate dallo specifico DBMS (es. GIS e sistemi basati sulla localizzazione).

2 Il Modello Relazionale

2.1 Concetti Generali

Il modello relazionale fu proposto da **E.F. Codd** (IBM Research) nel 1970, nel famoso articolo "*A Relational Model for Large Shared Data Banks*" (Communications of the ACM, giugno 1970). Il modello si basa sul concetto matematico di **relazione**, fondato sulla teoria degli insiemi. Questo lavoro fu alla base di una profonda rivoluzione nel campo delle basi di dati, che portò Codd a vincere il prestigioso **ACM Turing Award**.

2.2 Cos'è una Relazione

Informalmente, una **relazione** può essere vista come una tabella con un insieme di valori su ogni riga. Due livelli la definiscono:

- lo **schema** della relazione (livello *intensionale*);
- le **istanze** della relazione (livello *estensionale*).

2.2.1 Definizione Intensionale

Lo schema di una relazione definisce:

- il nome della relazione (es. STUDENTI);
- il nome di ogni attributo (es. SID, NOME, LOGIN, ...);
- il dominio di ogni attributo (es. INTEGER, STRING, ...), ovvero l'insieme dei valori che quell'attributo può assumere.

L'ordine degli attributi nello schema non ha alcun significato specifico. Notazione standard:

```
Students(sid: string, name: string, login: string, age: integer, gpa: real)
```

Il numero di attributi definisce il **grado** (o *arietà*) della relazione (5 nell'esempio sopra).

2.2.2 Definizione Estensionale

Un'**istanza** di uno schema di relazione è un insieme di tuple (o record), ognuna con lo stesso numero di campi dello schema:

- essendo un insieme, **non possono esserci duplicati**;
- l'ordine delle tuple non ha alcun valore specifico.

Il numero di istanze (tuple) definisce la **cardinalità** della relazione.

2.3 Stato di una Relazione

Lo stato di una relazione è un sottoinsieme del prodotto cartesiano dei domini dei suoi attributi. Data una relazione $R(A_1, A_2, \dots, A_n)$, uno specifico stato di R è definito come:

$$r(R) \subseteq \text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n)$$

ovvero $r(R) = \{t_1, t_2, \dots, t_n\}$ dove ogni t_i è una tupla, $t_i = \langle v_1, v_2, \dots, v_n \rangle$ e ogni v_j è un elemento di $\text{dom}(A_j)$. Nello stato di una relazione l'ordine delle tuple non conta (è un insieme!).

Esempio. Sia $R(A_1, A_2)$ con $\text{dom}(A_1) = \{0, 1\}$ e $\text{dom}(A_2) = \{a, b, c\}$. Il prodotto $\text{dom}(A_1) \times \text{dom}(A_2)$ è l'insieme di tutte le coppie ordinate possibili:

$$\{\langle 0, a \rangle, \langle 0, b \rangle, \langle 0, c \rangle, \langle 1, a \rangle, \langle 1, b \rangle, \langle 1, c \rangle\}$$

Un possibile stato $r(R)$ è $\{\langle 0, a \rangle, \langle 0, b \rangle, \langle 1, c \rangle\}$, con cardinalità 3.

2.4 Chiave di una Relazione

Ogni riga di una relazione ha un campo (o un insieme di campi) il cui valore identifica univocamente quella riga: si parla di **chiave** (*key*) della relazione. Talvolta si usano valori convenzionali per identificare una riga: si parla in questo caso di **chiavi artificiali** (*artificial key*) o **chiavi surrogate** (*surrogate key*).

2.5 Terminologia

Informale	Formale
Tabella	Relazione
Intestazione colonna	Attributo
Tutti i possibili valori di una colonna	Dominio
Riga della tabella	Tupla
Definizione della tabella	Schema della relazione
Tabella popolata	Stato della Relazione

Tabella 2: Corrispondenza tra terminologia informale e formale

2.6 Vincoli

I vincoli determinano quali stati di una relazione sono ammissibili. Sono di tre tipi:

1. **Vincoli impliciti:** dipendono dal data model stesso (es. il modello relazionale non ammette liste come valore di un attributo);
2. **Vincoli basati sullo schema** (o espliciti): definiti nello schema con gli strumenti forniti dal modello (es. un vincolo di integrità referenziale);
3. **Vincoli applicativi o semantici:** vanno oltre il potere espressivo del modello e devono essere imposti a livello di programma applicativo (es. un libro deve essere restituito entro 30 giorni dal prestito).

I principali vincoli espliciti nel modello relazionale sono:

- vincolo di dominio;
- vincolo di chiave;
- vincolo di integrità delle entità;
- vincolo di integrità referenziale.

2.7 Vincolo di Chiave

- **Superchiave** di R : insieme di attributi SK tale che non esistono due tuple distinte t_1, t_2 di $r(R)$ con $t_1[SK] = t_2[SK]$; la condizione deve valere in ogni stato valido di R .
- **Superchiave minimale** (o **chiave candidata**): una superchiave tale che rimuovendo un qualsiasi attributo si ottiene un insieme che non è più superchiave.
- Una chiave è sempre una superchiave, ma non viceversa.

Esempio. La relazione STUDENT ha due chiavi candidate: $\{sid\}$ e $\{login\}$. L'insieme $\{sid, name\}$ è una superchiave ma *non* una chiave (contiene una chiave, ma non è minimale). Ogni insieme di attributi che contiene una chiave è ovviamente una superchiave.

Se una relazione ha più chiavi candidate, una viene scelta (tipicamente dal DBA) come **chiave primaria**: in genere la più piccola per numero di attributi; quando ciò non è possibile, possono intervenire fattori soggettivi. I valori della chiave primaria identificano univocamente ogni tupla e possono essere usati per riferirsi ad essa da un'altra relazione.

2.8 Vincolo di Integrità dell'Entità (Entity Integrity)

Nessuno degli attributi che compongono la chiave primaria PK di una relazione R può avere valore NULL in alcuna tupla di $r(R)$:

$$t[PK] \neq null \quad \forall t \in r(R)$$

Se PK ha più attributi, NULL non è ammesso per nessuno di essi. È comunque possibile imporre che anche altri attributi (non chiave primaria) non possano assumere valore NULL.

2.9 Vincoli di Integrità Referenziale

A differenza degli altri, un vincolo di integrità referenziale coinvolge **due relazioni**: una relazione **referenziante** R_1 e una **referenziata** R_2 . In R_1 un insieme di attributi FK (**chiave esterna**, *Foreign Key*) fa riferimento alla chiave primaria PK di R_2 : una tupla $t_1 \in R_1$ riferenzia $t_2 \in R_2$ se $t_1[FK] = t_2[PK]$.

Esempio. `Enrolled` è la relazione referenziante, `Students` quella referenziata: `Enrolled.sid` è la chiave esterna che riferenzia `Students.sid`.

I valori della FK possono essere:

1. uno dei valori del corrispondente attributo PK in R_2 , oppure
2. il valore NULL (ma solo se la FK non fa parte della chiave primaria di R_1).

Osservazioni:

- la FK può fare riferimento alla *stessa* relazione a cui appartiene la PK;
- gli attributi della FK non devono necessariamente avere lo stesso nome dei corrispondenti attributi della PK.

Esempio: database COMPANY. Vincoli di integrità referenziale tipici in uno schema aziendale:

- `EMPLOYEE.Dno` $\rightarrow$ `DEPARTMENT.Dnumber`
- `EMPLOYEE.Super_ssn` $\rightarrow$ `EMPLOYEE.Ssn` (la FK riferenzia la stessa relazione della PK — vedi Osservazione 1)
- `DEPARTMENT.Mgr_ssn` $\rightarrow$ `EMPLOYEE.Ssn`
- `DEPT_LOCATIONS.Dnumber` $\rightarrow$ `DEPARTMENT.Dnumber`
- `PROJECT.Dnum` $\rightarrow$ `DEPARTMENT.Dnumber`
- `WORKS_ON.Essn` $\rightarrow$ `EMPLOYEE.Ssn`; `WORKS_ON.Pno` $\rightarrow$ `PROJECT.Pnumber`
- `DEPENDENT.Essn` $\rightarrow$ `EMPLOYEE.Ssn`

Esercizio 1

Dati gli schemi $R(A, B)$, $S(B, C)$ con Primary Key (A) e Foreign Key (B) references S(B), cosa si può concludere?

- a) A non può essere NULL ma B sì
- b) A e B possono essere NULL
- c) La primary key è parte della dichiarazione di R , la foreign key è nella dichiarazione di S
- d) B è una chiave primaria o una chiave candidata per S

Soluzione: d) — per definizione, una chiave esterna deve fare riferimento a una chiave primaria (o comunque candidata) della relazione referenziata, altrimenti il vincolo di integrità referenziale non sarebbe ben definito.

2.10 Schema di un Database Relazionale

I vincoli di integrità referenziale coinvolgono più di una relazione: non basta quindi considerare lo schema di una singola relazione. Uno **schema di database relazionale** è un insieme S di schemi di relazione appartenenti alla stessa base di dati:

$$S = \langle \{R_1, R_2, \dots, R_n\}, IC \rangle$$

dove $R_1, \dots, R_n$ sono i nomi degli schemi di relazione e IC è l'insieme dei vincoli di integrità.

2.11 Altri Tipi di Vincolo

I **vincoli di integrità semantica** si riferiscono al significato dell'applicazione e non possono essere espressi nel modello in sé (es. "il numero massimo di ore che un dipendente può lavorare su tutti i progetti è di 46 ore per settimana"). Serve quindi un linguaggio per la specifica di vincoli: SQL-99 introduce *trigger* e *asserzioni*, non implementati in modo uniforme da tutti i DBMS. In MySQL si può usare **CHECK**:

```
CREATE TABLE Person (...,  
    Age int CHECK (Age >= 18));
```

2.12 Stato di una Base di Dati Relazionale

Uno stato di una base di dati relazionale con schema S è un insieme di stati di relazione $\{r_1, r_2, \dots, r_m\}$ tale che ogni r_i è uno stato di R_i e soddisfa i vincoli di integrità in IC . Uno stato di una base di dati è talvolta chiamato **istantanea** (*snapshot*) o **istanza** (*instance*). Uno stato che non rispetta i vincoli in IC è uno **stato non valido**.

2.13 Operazioni di Aggiornamento e Violazione dei Vincoli

Le principali operazioni per cambiare lo stato di una base di dati sono:

- **INSERT**: inserimento di una nuova tupla;
- **DELETE**: cancellazione di una tupla;
- **MODIFY**: modifica di un attributo di una tupla.

Queste operazioni non devono mai portare alla violazione di un vincolo di integrità: garantire questa condizione può richiedere di propagare automaticamente gli aggiornamenti. Se si verifica una violazione, le opzioni possibili sono:

- **RESTRICT** (*NO ACTION*, *REJECT*): impedire l'operazione;
- **CASCADE**, **SET NULL**, **SET DEFAULT**: avviare altri aggiornamenti per eliminare la violazione;
- eseguire routine specifiche definite dall'utente.

2.13.1 Violazioni per Operazione INSERT

L'INSERT può violare **tutti** i vincoli:

- **dominio**: il valore di un attributo non appartiene al dominio specificato;
- **chiave**: inserimento di una tupla il cui valore di chiave esiste già;
- **integrità referenziale**: valore di FK che non esiste come valore di PK nella relazione referenziata;
- **integrità dell'entità**: valore della chiave primaria della nuova tupla è NULL.

Esercizio 2

Un'operazione di INSERT in una relazione può produrre la violazione di quale vincolo? (Dominio / Chiave / Integrità referenziale / Integrità delle entità / Nessuna / Tutte)

Soluzione: Tutte — come visto sopra, l'INSERT è l'unica operazione che può potenzialmente violare ciascuno dei quattro vincoli contemporaneamente.

2.13.2 Violazioni per Operazione DELETE

Il DELETE può violare **solo** vincoli di integrità referenziale, se si rimuove una tupla la cui chiave primaria è referenziata da altre tuple.

Esercizio 3

Dato lo schema `Enrolled(sid, cid, grade)` e `Students(sid, name, login, age, gpa)`, con `sid` chiave esterna di `Enrolled` verso `Students(sid)`: la cancellazione di una qualsiasi tupla nella tabella `ENROLLED` non può produrre la violazione di alcun vincolo. Vero o Falso?

Soluzione: Vero — `Enrolled` è la relazione "figlia" (referenziante): eliminare un suo record non viola mai l'integrità referenziale. La violazione scatterebbe solo cancellando un record "padre" in `Students` ancora referenziato da altre tuple.

2.13.3 Violazioni per Operazione UPDATE

L'UPDATE può violare tutti i vincoli, a seconda dell'attributo modificato:

- update della **chiave primaria**: integrità referenziale;
- update di una **chiave esterna**: integrità referenziale;
- update di un attributo ordinario: vincoli di dominio, o vincoli `UNIQUE/NOT NULL`.

2.14 Come Preservare l'Integrità Referenziale

- **RESTRICT** (*NO ACTION*): rifiuta l'operazione;
- **CASCADE**: cancella tutte le tuple che referenziavano la PK della tupla cancellata/modificata;

- **SET NULL**: assegna NULL alla FK delle tuple che referenziavano la tupla cancellata/modificata;
- **SET DEFAULT**: assegna un valore di default alla FK delle tuple che referenziavano la tupla cancellata/modificata.

Attenzione: le opzioni SET NULL e SET DEFAULT *non* sono disponibili se la chiave esterna include attributi che sono anche chiave primaria della tabella referenziante (altrimenti si violerebbe l'integrità dell'entità).

Esempio in SQL.

```
CREATE TABLE Enrolled (
    studid  CHAR(20),
    cid     CHAR(20),
    grade   CHAR(10),
    PRIMARY KEY (studid, cid),
    FOREIGN KEY (studid) REFERENCES Students
        ON DELETE CASCADE
        ON UPDATE NO ACTION
);
```

3 Algebra Relazionale

3.1 Introduzione

L'**algebra relazionale** è un linguaggio di interrogazione formale e **procedurale**: ogni operatore prende in input una o due relazioni e produce in output una relazione (proprietà di **chiusura**). Questo permette di comporre e annidare espressioni, applicando un operatore al risultato di un altro. L'algebra relazionale è alla base dei linguaggi pratici come SQL.

Operatori di base (un insieme completo, da cui derivano tutti gli altri):

- Selezione (σ), Proiezione (π);
- Unione ($\cup$), Differenza ($-$);
- Prodotto Cartesiano ($\times$);
- Ridenominazione (ρ).

Operatori derivati (esprimibili in funzione dei precedenti, ma usati per comodità):

- Intersezione ($\cap$);
- Join ($\bowtie$): theta-join, equijoin, join naturale;
- Divisione ($\div$).

3.2 Schema e Dati di Riferimento

Negli esempi che seguono si userà questo schema:

Sailors(sid: integer, sname: string, rating: integer, age: real)
 Boats(bid: integer, bname: string, color: string)
 Reserves(sid: integer, bid: integer, day: date)

con le seguenti istanze di esempio:

S1 (istanza di Sailors)				S2 (istanza di Sailors)			
sid	sname	rating	age	sid	sname	rating	age
22	dustin	7	45.0	28	yuppy	9	35.0
31	lubber	8	55.5	31	lubber	8	55.5
58	rusty	10	35.0	44	guppy	5	35.0
				58	rusty	10	35.0

R1 (istanza di Reserves)		
sid	bid	day
22	101	10/10/96
58	103	11/12/96

Tabella 3: Istanze di esempio usate negli operatori seguenti

3.3 Selezione (SELECT, σ)

L'operatore di **selezione** σ permette di selezionare un sottoinsieme delle tuple di una relazione, applicando a ciascuna di esse una formula booleana F .

Espressione: $\sigma_F(R)$	
Schema	$R(X) \rightarrow X$
Istanza	$r \rightarrow \sigma_F(r) = \{t \mid t \in r \text{ AND } F(t) = \text{vero}\}$

Tabella 4: Definizione formale di SELECT

F si compone di predicati connessi da AND ($\wedge$), OR ($\vee$) e NOT ($\neg$). Ogni predicato è del tipo $A \theta c$ oppure $A \theta B$, dove:

- A e B sono attributi in X ;
- $c \in \text{dom}(A)$ è una costante;
- θ è un operatore di confronto, $\theta \in \{=, \neq, <, >, \leq, \geq\}$.

Esempio. Trovare in S2 tutti i sailor che hanno giudizio superiore a 8: $\sigma_{\text{rating} > 8}(S2)$.

Risultato: $\{\langle 28, \text{yuppy}, 9, 35.0 \rangle, \langle 58, \text{rusty}, 10, 35.0 \rangle\}$ — stesso numero di attributi, selezionate le due (sole) tuple che soddisfano $\text{rating} > 8$.

3.4 Proiezione (PROJECT, π)

L'operatore di **proiezione** π estrae, da ogni tupla della relazione di input, solo gli attributi specificati, eliminando le colonne non richieste. Essendo il risultato un insieme, eventuali **tuple duplicate vengono eliminate**: per questo il numero di tuple del risultato può essere inferiore a quello della relazione di partenza.

Esempio. Trovare nome e giudizio di tutti i sailor (istanza S2): $\pi_{sname, rating}(S2)$.

Risultato: $\{\langle yuppy, 9 \rangle, \langle lubber, 8 \rangle, \langle guppy, 5 \rangle, \langle rusty, 10 \rangle\}$.

3.5 Unione, Differenza e Intersezione

Questi operatori insiemistici richiedono che le due relazioni operande siano **union-compatibili**: stesso numero di attributi, con domini corrispondenti compatibili.

- $R \cup S$: tutte le tuple che appartengono a R o a S (o entrambe);
- $R - S$: le tuple che appartengono a R ma non a S ;
- $R \cap S$ (*derivato*): le tuple che appartengono sia a R sia a S ; $R \cap S = R - (R - S)$.

$\cup$ e $\cap$ sono **commutativi**; $-$ **non** lo è.

3.6 Prodotto Cartesiano ($\times$)

$R \times S$ combina **ogni** tupla di R con **ogni** tupla di S : lo schema del risultato è la concatenazione degli attributi di R e di S (con $|R| \cdot |S|$ tuple in output). Se R e S hanno un attributo con lo stesso nome, questo compare due volte nello schema risultato, creando ambiguità.

Esempio. $S1 \times R1$: 6 tuple (3×2), con la colonna **sid** duplicata (una da S1, una da R1) — l'ambiguità che motiva l'operatore di ridenominazione.

3.7 Ridenominazione (RENAME, ρ)

L'operatore ρ rinomina la relazione e/o i suoi attributi, senza modificare i dati, per risolvere ambiguità come quella del prodotto cartesiano.

Esempio. $\rho(C(1 \rightarrow sid1, 5 \rightarrow sid2), S1 \times R1)$: rinomina il 1° attributo (sid da S1) in **sid1** e il 5° (sid da R1) in **sid2**.

3.8 Join

3.8.1 Theta-Join

Il **theta-join** $R \bowtie_c S$ equivale a $\sigma_c(R \times S)$: un prodotto cartesiano seguito da una selezione secondo una condizione c qualsiasi (non solo uguaglianze). Lo schema del risultato mantiene **entrambi** gli attributi coinvolti nella condizione, anche se omonimi.

Esempio. $S1 \bowtie_{S1.sid < R1.sid} R1$: delle 6 combinazioni possibili, solo 2 soddisfano la condizione $sid_{S1} < sid_{R1}$: $\langle 22, dustin, 7, 45.0, 58, 103, 11/12/96 \rangle$ e $\langle 31, lubber, 8, 55.5, 58, 103, 11/12/96 \rangle$.

3.8.2 Join Naturale

Espressione: $R_1 \bowtie R_2$	
Schema	$R_1(X_1), R_2(X_2) \rightarrow X_1 \cup X_2$
Istanza	$r_1, r_2 \rightarrow r_1 \bowtie r_2 = \{t \mid t[X_1] \in r_1 \text{ AND } t[X_2] \in r_2\}$

Tabella 5: Definizione formale del join naturale

Ogni tupla del risultato combina (*match*) una tupla di r_1 con una di r_2 sulla base dell'**uguaglianza** dei valori degli attributi comuni ($X_1 \cap X_2$); questi attributi compaiono **una sola volta** nello schema risultato.

Osservazioni:

- una tupla che non trova corrispondenza nell'altra relazione si dice **dangling**;
- la cardinalità del join è compresa tra 0 e $|r_1| \cdot |r_2|$;
- se il join è eseguito su una superchiave di $R_1(X_1)$, allora $|r_1 \bowtie r_2| \leq |r_2|$;
- se $X_1 \cap X_2$ è chiave primaria di $R_1(X_1)$ e chiave esterna in $R_2(X_2)$ (vincolo di integrità referenziale), allora $|r_1 \bowtie r_2| = |r_2|$.

3.8.3 Equijoin

È la forma di join più comunemente utilizzata: la condizione c include **soltanto** confronti di uguaglianza (=).

Esempio. $S1 \bowtie_{sid=sid} R1$: risultato $\{\langle 22, \text{dustin}, 7, 45.0, 101, 10/10/96 \rangle, \langle 58, \text{rusty}, 10, 35.0, 103, 11/12/96 \rangle\}$ (la tupla con $sid = 31$ è *dangling*: nessuna corrispondenza in R1).

3.9 Divisione ($\div$)

Dati $A(x, y)$ e $B(y)$, $A \div B$ restituisce l'insieme delle tuple x tali che, per **ogni** tupla y in B , la coppia (x, y) compare in A . Caso d'uso tipico: "trovare i sailor che hanno prenotato **tutte** le barche in un certo insieme".

Esempio. Dato $A(sno, pno)$ con le coppie $\{s_1p_1, s_1p_2, s_1p_3, s_1p_4, s_2p_1, s_2p_2, s_3p_2, s_4p_2, s_4p_4\}$:

- $B_1 = \{p_2\} \Rightarrow A/B_1 = \{s_1, s_2, s_3, s_4\}$ (tutti prenotano p_2);
- $B_2 = \{p_2, p_4\} \Rightarrow A/B_2 = \{s_1, s_4\}$ (solo loro prenotano **sia** p_2 **sia** p_4);
- $B_3 = \{p_1, p_2, p_4\} \Rightarrow A/B_3 = \{s_1\}$ (solo s_1 prenota tutte e tre).

3.10 Outer Join

A differenza del join (naturale o theta), l'**outer join** conserva anche le tuple *dangling*, riempiendo con NULL gli attributi mancanti:

- **LEFT OUTER JOIN**: conserva le tuple *dangling* della relazione di sinistra;
- **RIGHT OUTER JOIN**: conserva le tuple *dangling* della relazione di destra;
- **FULL OUTER JOIN**: conserva le tuple *dangling* di entrambe le relazioni.

3.11 Espressioni ed Alberi di Query

Un'espressione di algebra relazionale può essere rappresentata come un **albero di query**: le foglie sono le relazioni di base, i nodi interni sono gli operatori, e la valutazione procede dal basso verso l'alto. Espressioni algebricamente equivalenti (stesso risultato per ogni istanza valida) possono corrispondere ad alberi diversi con costi di esecuzione molto differenti: è la base per l'**ottimizzazione delle query**, approfondita quando si tratterà l'analisi dei costi.

4 SQL

4.1 Cos'è SQL

SQL (*Structured Query Language*) è un linguaggio specializzato per la gestione di basi di dati relazionali: standardizzato e ampiamente utilizzato, permette la creazione, modifica e interrogazione dei dati. Si divide in:

- **DDL** (*Data Definition Language*): definisce la struttura della base di dati (creazione tabelle, modifiche allo schema, definizione di vincoli);
- **DML** (*Data Manipulation Language*): manipola i dati (inserimento, aggiornamento, cancellazione, interrogazione).

4.2 Istanza di Riferimento

Negli esempi che seguono si usa questa istanza (tratta dal libro di testo):

Sailors (istanza S3)				Boats (istanza B1)		
sid	sname	rating	age	bid	bname	color
22	Dustin	7	45.0	101	Interlake	blue
29	Brutus	1	33.0	102	Interlake	red
31	Lubber	8	55.5	103	Clipper	green
32	Andy	8	25.5	104	Marine	red
58	Rusty	10	35.0			
64	Horatio	7	35.0			
71	Zorba	10	16.0			
74	Horatio	9	35.0			
85	Art	3	25.5			
95	Bob	3	63.5			

Tabella 6: Istanze di riferimento per gli esempi SQL (**Reserves**, istanza R2, omessa qui: 10 tuple, 4 per sid 22, 3 per sid 31, 2 per sid 64, 1 per sid 74, verificabili dal risultato della query in Sez. 4.9.2)

4.3 Struttura Base di una Query

```
SELECT  [DISTINCT] target-list
FROM    relation-list
WHERE   qualification
```

- **relation-list**: lista di nomi di relazioni (talvolta con *variabili di range*) — da 1 a n ;
- **target-list**: lista di attributi che appartengono alle relazioni nella relation-list;
- **qualification**: predicati di confronto, eventualmente combinati con AND, OR o NOT;
- **DISTINCT**: chiave opzionale per eliminare i duplicati dal risultato (il **default** è che **NON** vengono eliminati).

4.3.1 Semantica Valutativa (concettuale)

1. si calcola il prodotto cartesiano delle tabelle nella relation-list;
2. si scartano le tuple che non soddisfano la qualification;
3. si eliminano le colonne degli attributi che non appartengono alla target-list;
4. se presente DISTINCT, si eliminano i duplicati.

Nota: questa è una strategia concettuale, non quella realmente usata dal DBMS — è molto inefficiente; il modulo di *query optimization* costruisce piani di esecuzione più efficienti ma semanticamente equivalenti.

Esempio.

```
-- Q0: sname ed eta' di tutti i marinai
select s.sname, s.age
from sailors s;
```

```
-- Q0.1: sname ed eta' dei marinai con piu' di 50 anni
select s.sname, s.age
from sailors s
where s.age > 50;
```

Risultato di Q0.1 sull'istanza S3: $\{ \langle \text{Lubber}, 55.5 \rangle, \langle \text{Bob}, 63.5 \rangle \}$ — le uniche due tuple con $age > 50$.

4.4 Operatori di Confronto

- aritmetici: =, <, >, >=, <=, <>;
- BETWEEN: valore compreso in un intervallo;
- IN: valore compreso in una lista;
- LIKE: confronto fra stringhe con caratteri jolly (% e _);
- IS [NOT] NULL: verifica se un valore ha (o non ha) valore NULL.

Attenzione: WHERE $x = \text{NULL}$ è **sempre sbagliato** — va usato WHERE $x \text{ IS NULL}$.

Esercizio 1

Quale dei seguenti operatori è usato per verificare se il valore di un attributo è uguale a NULL in SQL? (nessuna delle opzioni / IS / IN / =)

Soluzione: IS (cioè IS NULL) — l'operatore = non funziona mai con NULL, per via della logica a tre valori di SQL (vedi Esercizio 2).

4.5 Operatori di Aggregazione

SQL supporta 5 operatori di aggregazione, che vanno oltre gli operatori dell'algebra relazionale:

- `count(*)`: numero di tuple restituite da una query;
- `count([DISTINCT] A)`: numero di valori (unici) di A;
- `sum([DISTINCT] A)`: somma dei valori (unici) di A;
- `avg([DISTINCT] A)`: media dei valori (unici) di A;
- `MAX(A) / MIN(A)`: valore più alto / più basso della colonna A.

Nota: gli operatori di aggregazione ignorano i valori NULL nella colonna aggregata (eccetto `count(*)`, che conta le tuple indipendentemente dai NULL).

4.6 GROUP BY e HAVING

Quando serve applicare l'aggregazione non a tutte le tuple ma a **gruppi** di tuple (es. "quante volte è stata noleggiata ogni barca", "età media dei marinai per ogni rating"), si estende la struttura base:

```
SELECT  [DISTINCT] select-list
FROM    from-list
WHERE   qualification
GROUP BY grouping-list
HAVING  group-qualification
```

- la **select-list** è composta da (1) nomi di colonne e (2) termini **agg-op(colonna) AS nuovo_nome**;
- ogni colonna in (1) **deve** comparire anche nella **grouping-list** (altrimenti potrebbe avere valori diversi nello stesso gruppo);
- le espressioni nella **group-qualification** devono avere un solo valore per gruppo;
- senza GROUP BY, l'intera tabella è considerata un singolo gruppo.

Valutazione (concettuale):

1. prodotto cartesiano delle tabelle nel FROM;

2. applicazione del WHERE;
3. eliminazione delle colonne non richieste (non in select-list/grouping-list/group-qualification);
4. riordino in base al GROUP BY, per identificare i gruppi;
5. applicazione della group-qualification nella clausola HAVING;
6. generazione di una riga di risposta per ogni gruppo rimanente.

4.6.1 Esempio (Q32)

Trovare, per ogni livello di rating con almeno due marinai maggiorenni ($age \geq 18$), l'età del più giovane tra questi.

```
SELECT  S.rating, MIN(S.age) AS minage
FROM    Sailors S
WHERE   S.age >= 18
GROUP BY S.rating
HAVING  COUNT(*) > 1
```

Sull'istanza S3: il WHERE esclude Zorba (16.0, minorenne). Raggruppando per rating: rating 1 (Brutus, 1 sailor), rating 3 (Art 25.5, Bob 63.5 — 2 sailor), rating 7 (Dustin 45.0, Horatio 35.0 — 2 sailor), rating 8 (Lubber 55.5, Andy 25.5 — 2 sailor), rating 9 (Horatio 35.0, 1 sailor), rating 10 (Rusty 35.0, 1 sailor). L'HAVING scarta i gruppi con un solo sailor. **Risultato:** $\{\langle 3, 25.5 \rangle, \langle 7, 35.0 \rangle, \langle 8, 25.5 \rangle\}$.

4.7 Interrogazioni Annidate (Nested Queries)

Una **query annidata** (o *nested*) è una query (*interna*) che compare dentro un'altra (*esterna*). Semanticamente è simile a un *nested loop*: per ogni tupla della query esterna, si valuta il WHERE eseguendo la query interna. Una query annidata può restituire:

- un **singolo valore**, da confrontare con un attributo della query esterna;
- un **insieme di valori**, tipicamente con IN, EXISTS, ANY o ALL.

4.7.1 Query Indipendenti e Correlate

- **Indipendente:** eseguibile da sola, senza riferimenti a colonne/valori della query esterna; viene eseguita **una sola volta**.
- **Correlata:** la query interna fa riferimento a colonne della query esterna; deve essere **rieseguita per ogni riga** della query esterna.

4.7.2 Casi d'Uso

- filtrare con un valore calcolato dinamicamente (es. marinai con rating minore del rating medio);
- confrontare con un risultato derivato (es. il marinaio con il rating più alto);

- verificare l'esistenza (o assenza) di certi valori (**EXISTS**; es. marinai che non hanno mai prenotato);
- aggregazione condizionale (es. rating dei marinai con meno di 2 prenotazioni);
- unire il risultato di una subquery con altre tabelle via **JOIN**.

4.7.3 Annidamento a Più Livelli e Operatori **ANY/ALL**

È possibile annidare più di un livello. Con **ANY**: la condizione è vera se vale per **almeno una** tupla del risultato della subquery (se la subquery è vuota, **ANY** restituisce **FALSE**). Con **ALL**: la condizione deve valere per **tutte** le tuple (se la subquery è vuota, **ALL** restituisce **TRUE** — *vacuous truth*).

4.8 Operatori Insiemistici

- **UNION** (o **UNION ALL**): unione di due insiemi di tuple compatibili; **UNION** elimina i duplicati, **UNION ALL** no;
- **INTERSECT**: tuple comuni a entrambi gli insiemi;
- **EXCEPT**: tuple del primo insieme che non compaiono nel secondo (differenza insiemistica).

Le subquery possono comparire in diversi punti di una query: nel **FROM** (per costruire una tabella temporanea), o nel **WHERE** (per calcolare al volo un valore o un insieme con cui confrontare).

4.9 JOIN

Le 4 forme principali:

1. **(INNER) JOIN**: solo le tuple che soddisfano la condizione in **entrambe** le tabelle;
2. **LEFT (OUTER) JOIN**: tutte le tuple della tabella di sinistra, più quelle di destra che soddisfano la condizione (**NULL** altrimenti);
3. **RIGHT (OUTER) JOIN**: simmetrico, tutte le tuple della tabella di destra;
4. **FULL (OUTER) JOIN**: tutte le tuple di entrambe le tabelle a fronte di un match in almeno una¹.

4.9.1 **INNER JOIN** vs. **Prodotto Cartesiano + WHERE**

Un **INNER JOIN** si può scrivere in due modi equivalenti: con l'operatore esplicito **JOIN . . . ON**, oppure mettendo le tabelle nel **FROM** e la condizione nel **WHERE** — restituiscono **esattamente lo stesso risultato**. La forma **JOIN . . . ON** è raccomandata per chiarezza (separa la condizione di join dagli altri filtri), ma nella pratica sono intercambiabili. **Non** lo sono, invece, quando serve un **OUTER JOIN**.

¹MariaDB (come molti altri DBMS) **non supporta** il **FULL OUTER JOIN**.

4.9.2 Esempio di LEFT JOIN

Quante prenotazioni ha fatto ogni marinaio (inclusi quelli con zero prenotazioni)?

```
select s.sid, count(r.sid)
from sailors s left join reserves r
on s.sid = r.sid
group by s.sid;
```

Con un INNER JOIN o un prodotto cartesiano, i marinai senza prenotazioni (dangling) sparirebbero dal risultato: serve il LEFT JOIN per conservarli con conteggio 0. **Risultato** su S3/R2: $\langle 22, 4 \rangle, \langle 29, 0 \rangle, \langle 31, 3 \rangle, \langle 32, 0 \rangle, \langle 58, 0 \rangle, \langle 64, 2 \rangle, \langle 71, 0 \rangle, \langle 74, 1 \rangle, \langle 85, 0 \rangle, \langle 95, 0 \rangle$.

4.9.3 ANTI JOIN

I casi che restituiscono solo le tuple **senza** corrispondenza nell'altra relazione si chiamano LEFT/RIGHT/FULL **ANTI JOIN**. Poco usati nello sviluppo applicativo, ma utili in business intelligence/data science (es. "tutti i marinai che non hanno mai noleggiato una barca").

4.10 Linee Guida Pratiche per Scrivere Query SQL

1. **Studia lo schema:** comprendi tabelle/attributi, domini, vincoli di integrità referenziale (serviranno per i JOIN);
2. **Leggi bene la richiesta:** individua gli attributi in output ($\rightarrow$ target-list), la tabella di provenienza di ciascuno ($\rightarrow$ relation-list), i filtri necessari;
3. **Cerca le aggregazioni:** parole come "valore medio" ($\rightarrow$ AVG), "il totale" ($\rightarrow$ SUM), "il numero di" ($\rightarrow$ COUNT) suggeriscono operatori di aggregazione, eventualmente con GROUP BY/HAVING;
4. **Comprendi i vincoli di integrità referenziale:** individua gli attributi per il JOIN (tipicamente PK $\rightarrow$ FK) e il tipo di JOIN semanticamente corretto (INNER, OUTER, ANTI, ...) — ricorda che il WHERE opera tupla per tupla;
5. **Cerca le query annidate:** confronto con valore dinamico, appartenenza a un insieme ($\rightarrow$ IN), confronto con insieme di valori ($\rightarrow$ ANY/ALL), verifica di esistenza ($\rightarrow$ EXISTS);
6. **Individua eventuali operazioni insiemistiche:** unione di tuple con stesso schema da tabelle diverse ($\rightarrow$ UNION), tuple comuni ($\rightarrow$ INTERSECT), tuple in un insieme ma non nell'altro ($\rightarrow$ EXCEPT);
7. **Verifica la coerenza semantica del risultato:** scomponi la query in parti più semplici, controlla filtri/cardinalità/colonne restituite, non dimenticare DISTINCT o l'ordinamento se richiesti.

Esercizio 2

Si assuma che la relazione $R(A, B)$ abbia le dichiarazioni PRIMARY KEY (A) e UNIQUE (B). Quale delle seguenti affermazioni è vera? (A può essere NULL, B no / Sia A che B possono essere NULL / A e B non possono essere NULL / A non può essere NULL, B sì)

Soluzione: A non può essere NULL, B sì — il vincolo PRIMARY KEY implica sempre NOT NULL (integrità dell'entità), mentre UNIQUE vieta solo i duplicati tra valori non-NULL: la maggior parte dei DBMS ammette (tipicamente) un valore NULL in una colonna UNIQUE, proprio perché NULL non è mai considerato uguale a un altro NULL.

Esercizio 3

Data la tabella *R*:

A	B	C
1	2	6
NULL	NULL	3
4	NULL	NULL
5	6	NULL
NULL	NULL	NULL

Qual è il risultato di `select C from R where A=B?` (l'insieme vuoto / NULL / 3, NULL / 3)

Soluzione: l'insieme vuoto. Per la logica a tre valori di SQL, $A=B$ è vero solo se **entrambi** non sono NULL e uguali. Riga 1: $1 = 2$ falso. Righe 2 e 5: $NULL=NULL \rightarrow$ UNKNOWN (non vero). Riga 3: $4 = NULL \rightarrow$ UNKNOWN. Riga 4: $5 = 6$ falso. Nessuna riga soddisfa la condizione: il WHERE non include mai le righe con esito UNKNOWN, solo quelle con esito TRUE.

5 Progettazione Concettuale: il Modello Entity-Relationship (ER)

5.1 La Progettazione di una Base di Dati

Le fasi principali della progettazione di una BdD sono:

1. Analisi dei requisiti;
2. Progettazione concettuale;
3. Progettazione logica;
4. Raffinamento dello schema logico (**normalizzazione**);
5. Progettazione del livello fisico;
6. Progettazione applicativa e sicurezza.

Per le fasi 1-3 ci si chiede *quali dati* devono essere raccolti; per le fasi 4-6, *quali applicazioni* useranno questi dati e quali saranno le operazioni più frequenti/costose.

Un **modello** è una rappresentazione semplificata di una porzione di mondo (*mini-mondo*); è *epistemologicamente adeguato* se permette di esprimere concretamente i fatti noti del mini-mondo di interesse (McCarthy & Hayes, 1969). Per le basi di dati, il modello più usato per la progettazione concettuale è il modello **Entity-Relationship (ER)**.

5.2 Concetti Fondamentali: Entità e Attributi

- **Entità** (*entity*): un oggetto del mondo reale (o del mini-mondo), distinguibile dagli altri, che vogliamo rappresentare nella BD (es. il dipendente John Smith).
- **Insieme di entità** (*entity set*): a livello estensionale, un insieme di entità simili (stessi attributi).
- **Tipo di entità** (*entity type*): la definizione a livello intensionale delle entità che possono appartenere a un entity set di quel tipo.
- **Attributi** (*attributes*): le proprietà usate per descrivere un'entità (es. nome, SSN, indirizzo). Ogni attributo è associato a un dominio (*value set*).

5.2.1 Tipi di Attributo

- **Semplice** (*simple*): un singolo valore atomico (es. 'M' per Sesso);
- **Composto** (*composite*): più componenti (es. Indirizzo $\rightarrow$ Via, Città, CAP, ...); un componente di un attributo composto può a sua volta essere composto;
- **Multi-valore** (*multi-valued*): un'entità può avere più valori per lo stesso attributo (es. CERTIFICAZIONI di un dipendente).

5.2.2 Attributi Chiave

Ogni entity type ha un attributo (o insieme di attributi) che identifica univocamente ogni entità: l'**attributo chiave**. Può essere:

- **composto** (es. per un'AZIENDA: RAGIONE_SOCIALE + PROVINCIA);
- **non unico**: un entity type può avere più **chiavi candidate** (es. AUTOMOBILE: NumeroTelaio/VIN oppure Targa); una viene scelta come **chiave primaria**.

In ER, la chiave primaria è sempre sottolineata.

5.2.3 Rappresentazione Grafica

- un **entity type** è un **rettangolo** con il nome del tipo;
- gli **attributi** sono **ovali**, collegati al loro entity type; gli attributi chiave sono sottolineati.

5.3 Relazioni (Relationships)

Una **relazione** (*relationship*) è un'associazione tra due o più entità (es. la relazione tra un IMPIEGATO e il DIPARTIMENTO per cui lavora). Relazioni dello stesso tipo sono raggruppate in un **relationship type**.

5.3.1 Grado di una Relazione

Il numero di entity types che partecipano a un relationship type è il suo **grado**: una relazione tra 2 entity types è **binaria** (grado 2); tra 3, **ternaria** (grado 3) — es. IMPIEGATO-DIPARTIMENTO-SEDE, quando un impiegato lavora per un dipartimento che ha più sedi.

5.3.2 Attributi della Relazione

Una relazione può avere attributi descrittivi propri (es. **since** su `LAVORA_PER`), che descrivono la relazione stessa, non le entità partecipanti. **Attenzione:** una relazione è identificata unicamente dalle entità che vi partecipano, **non** dai suoi attributi descrittivi — quindi per ogni coppia $\langle ssn, did \rangle$ di `WORKS_IN` non può esistere più di un valore per **since** (se serve registrare più periodi, serve un modello diverso — vedi Sez. 5.10.2).

5.3.3 Relationship Type vs. Relationship Set

- **Relationship Type:** lo schema che descrive la relazione — nome, entity types partecipanti, vincoli;
- **Relationship Set:** l'insieme di istanze della relazione rappresentate nella BD in un certo istante (lo stato attuale del relationship type).

5.4 Vincoli sulle Relazioni

Tre tipi principali: vincoli di **chiave**, di **cardinalità**, di **partecipazione** (o **esistenza**).

5.4.1 Vincolo di Chiave (key constraint)

Esprime che un'entità può partecipare al **massimo una volta** in una relazione (es. un Dipartimento può avere al massimo un manager, ma non viceversa). Si generalizza a relazioni di grado > 2 : una certa entità non può comparire più di una volta nel relationship set. **Notazione:** freccia semplice dall'entità vincolata verso la relazione.

5.4.2 Vincoli di Cardinalità (cardinality ratio)

Specificano il numero massimo di volte in cui un'entità può comparire in un relationship set:

- **One-to-one (1:1)** — es. `PERSONA commissario_tecnico NAZIONALE`;
- **One-to-many / Many-to-one (1:N / N:1)** — es. `STUDENTE iscritto SCUOLA_DI_DOTTORATO`;
- **Many-to-many (M:N)** — es. `STUDENTE frequenta CORSO`.

5.4.3 Vincolo di Partecipazione (participation constraint)

Specifica quante volte un'entità deve comparire **al minimo** in una relazione (da cui anche il nome *Existence Dependency Constraint*):

- **partecipazione parziale:** l'entità può non comparire → **linea semplice**;
- **partecipazione totale:** l'entità deve comparire almeno una volta → **linea in grassetto**.

Esempio: `DEPARTMENTS` ha partecipazione totale in `MANAGES` (ogni dipartimento ha necessariamente un manager); `DEPARTMENTS` ed `EMPLOYEES` hanno entrambi partecipazione totale in `WORKS_IN` (ogni impiegato afferisce ad almeno un dipartimento, e viceversa).

5.4.4 Notazione (min, max)

Alternativa più precisa: ogni entità in E partecipa in almeno **min** e al massimo **max** istanze di R . Il default (assenza di vincoli espliciti) è **min=0**, **max=N** e non va indicato; si usa la notazione esplicita quando **min** > 1 o **max** è diverso da 1 o N (es. 5).

5.5 Relazioni Ricorsive

Una relazione può legare entità dello **stesso tipo** ma in **ruoli distinti**: es. REPORTS_TO, dove EMPLOYEE partecipa due volte con ruoli diversi (Supervisore e Subordinato).

5.6 Entity Types Deboli

Un'entità **debole** non ha un proprio attributo chiave sufficiente all'identificazione univoca, e dipende da un'altra entità (**identificante**) tramite una **relazione identificante**. È identificata dalla combinazione di:

- una **chiave parziale** (propria dell'entità debole);
- la chiave dell'entità identificante.

Esempio 1. DEPENDENT (familiare a carico) è identificato dal proprio nome (chiave parziale) + l'EMPLOYEE a cui è associato tramite la relazione identificante POLICY.

Esempio 2. ORDER_DETAILS (righe di un ordine, con prodotto e quantità) è identificato da OrderID (dell'ORDER a cui appartiene, tramite la relazione identificante INCLUDE) + ProductID.

5.7 Gerarchie di Classi (ISA)

Talvolta è utile organizzare entity types in gerarchie, per:

- **Generalizzazione**: due entity types vengono ricompresi in uno più generale;
- **Specializzazione**: un entity type viene suddiviso in sottoinsiemi con caratteristiche comuni (può avvenire su più dimensioni, creando gerarchie multilivello).

Esempio: EMPLOYEE specializzato in HOURLY_EMPS e CONTRACT_EMPS (generalizzati in EMPLOYEES).

Perché introdurre gerarchie ISA:

- per aggiungere attributi specifici di una sottoclasse (es. costo orario per impiegati a ore);
- per identificare il sottoinsieme di entità che partecipa a una certa relazione (es. solo i dipendenti a contratto in un circolo ricreativo aziendale).

5.7.1 Vincoli: Overlap e Coverage

- **Overlap**: possono due sottoclassi avere entità in comune? Es. `CONTRACT_EMPS` e `HOURLY_EMPS` **non** possono sovrapporsi; `CONTRACT_EMPS` e `SENIOR_EMPS` sì (si scrive `CONTRACT_EMPS OVERLAPS SENIOR_EMPS`). **Default: nessun overlap.**
- **Coverage** (completezza): le sottoclassi coprono tutta la superclasse? Es. `TIGRI` e `PANTERE` **non** coprono tutti i `FELINI`; `JUNIOR_EMPS` e `SENIOR_EMPS` presumibilmente coprono `EMPLOYEES` (si scrive `JUNIOR_EMPS AND SENIOR_EMPS COVER EMPLOYEES`). **Default: coverage non vale.**

Si hanno quindi 4 combinazioni: (no overlap, copertura totale), (no overlap, copertura parziale), (overlap, copertura totale), (overlap, copertura parziale). **Nota:** la generalizzazione è di solito (ma non necessariamente) totale, poiché le superclassi sono derivate dalle sottoclassi.

5.8 Aggregazione

Un relationship set normalmente associa entity set. L'**aggregazione** permette di associare un entity set con un **intero relationship set** (che a sua volta associa 2 o più entity set), trattandolo come un'entità unica ai fini di un'ulteriore relazione.

Esempio. Un impiegato (`EMPLOYEE`) può `MONITORARE` un `PROGETTO` (`PROJECT`) che è `SPONSORIZZATO` da uno o più `DIPARTIMENTI` (`DEPARTMENT`): `MONITORS` aggrega la relazione `SPONSORS` (tra `PROJECT` e `DEPARTMENT`).

5.9 Dai Requisiti al Modello: Regole Empiriche

1. **Analisi del linguaggio naturale:** nomi $\rightarrow$ entità candidate; verbi $\rightarrow$ relazioni candidate; aggettivi/frasi descrittive $\rightarrow$ attributi candidati;
2. **Chiedere chiarimenti al cliente: non inventare** — se un attributo o vincolo non è esplicitato, va discusso col cliente prima di aggiungerlo;
3. **Identificazione delle entità:** nomi concreti (Cliente, Prodotto, Ordine); generalizzare sostantivi simili/correlati in una categoria comune;
4. **Individuazione delle relazioni:** frasi come "un cliente effettua un ordine"; la **cardinalità** emerge da frasi come "ogni cliente può effettuare più ordini"; la **partecipazione** da frasi come "ogni dipartimento deve avere un manager" (totale);
5. **Identificazione degli attributi:** descrittori delle entità; le chiavi primarie da frasi come "ogni cliente è identificato con un ID", o dedotte dalla semantica;
6. **Validazione:** completezza (tutte le entità/relazioni richieste sono presenti) e assenza di **ridondanza** ingiustificata (es. duplicare il nome del cliente sia nell'anagrafica sia negli ordini);
7. **Iterazione e raffinamento:** presentare il diagramma ER al cliente, raccogliere feedback, iterare.

5.10 Scelte di Progettazione Concettuale con ER

Definire un modello ER non è un processo lineare: lo stesso insieme di requisiti può essere modellato in modi diversi, e non tutti i vincoli di dominio sono esprimibili in ER (limiti di espressività).

5.10.1 Entità vs. Attributo

Quando la scelta non è vincolante, si preferisce l'attributo per semplicità. Serve un **entity type** quando:

1. potremmo dover registrare **più valori** per la stessa entità (es. più indirizzi per lo stesso dipendente);
2. vogliamo **scomporre** ulteriormente l'informazione (es. indirizzo in via/comune/provincia per query mirate).

Stesso ragionamento per un attributo su una *relazione*: se **from/to** sono attributi di WORKS_IN, si può registrare un solo periodo per coppia impiegato-dipartimento; per registrarne più di uno serve un nuovo entity type.

5.10.2 Entità vs. Relazione

Esempio: un impiegato che gestisce un dipartimento ha anche un budget. Se il budget è associato alla relazione MANAGES, e un impiegato può gestire più dipartimenti con un budget **complessivo** (non per dipartimento), si presentano due problemi:

- **ridondanza**: il valore di DBUDGET si ripeterebbe identico per ogni istanza che lega l'impiegato ai vari dipartimenti (si riprenderà nella normalizzazione);
- **semantica scorretta**: si suggerisce che il budget appartenga alla relazione, mentre appartiene al manager.

Soluzione: specializzare EMPLOYEES con una sottoclasse MANAGERS, e assegnare a quest'ultima gli attributi SINCE e DBUDGET (se SINCE deve invece variare per dipartimento gestito, va lasciato sulla relazione).

5.10.3 Relazioni Binarie o Ternarie

Quando 3 entity types sono coinvolti in un requisito, bisogna chiedersi se si tratta davvero di un'unica relazione a 3 vie o di due relazioni binarie distinte.

Esempio. EMPLOYEE può possedere più POLICY (polizze), ognuna posseduta da un solo EMPLOYEE; ogni DEPENDENT può avere più POLICY. Vincoli aggiuntivi: una polizza non può essere posseduta congiuntamente da più impiegati; ogni polizza deve essere posseduta da un impiegato.

- **Ipotesi 1 (relazione ternaria con vincoli)**: un vincolo di chiave da POLICIES verso la relazione ternaria implicherebbe (erroneamente) che una polizza copra un solo DEPENDENT; un vincolo di partecipazione totale su POLICY funzionerebbe solo se ogni polizza copre almeno un DEPENDENT.

- **Ipotesi 2 (due relazioni binarie):** soluzione preferibile — evidenzia le due relazioni distinte a cui partecipa POLICIES, e usa una relazione binaria come identificante per l'entità debole (richiesto in alcune varianti di ER).

Non c'è una risposta univoca: dipende dalla semantica dei requisiti (nell'esempio dei manager con budget visto sopra, invece, la scelta corretta era la relazione ternaria).

5.10.4 Aggregazione o Relazione Ternaria?

Riprendendo l'esempio MONITORS/SPONSORS: una relazione ternaria unica (SPONSORS2, tra EMPLOYEE-PROJECT-DEPARTMENT) sarebbe stata sufficiente solo se non fosse stato necessario un attributo **until** specifico su MONITORS; inoltre, non avrebbe permesso di esprimere facilmente il vincolo "ogni sponsorizzazione è monitorata al massimo da un impiegato", che con l'aggregazione (vincolo di chiave da MONITORS verso SPONSORS) si esprime naturalmente.

Esercizio 1

In ER un'aggregazione è definita come: (un operatore per aggregare entità in relazione gerarchica IS-A / un attributo dinamico ottenuto da operatori di aggregazione come somma o media / un modo per rappresentare una relazione tra un'entità e un gruppo di entità correlate, trattando questo gruppo come un'unica entità / nessuna delle opzioni)

Soluzione: un modo per rappresentare una relazione tra un'entità e un gruppo di entità correlate, trattando questo gruppo come un'unica entità — è la definizione esatta vista sopra (associare un entity set a un intero relationship set).

6 Mapping dal Modello ER al Modello Relazionale

6.1 Obiettivi del Mapping

- preservare il più possibile la conoscenza rappresentata nel modello concettuale;
- mantenere i vincoli, per quanto possibile: **alcuni vincoli non sono esprimibili** nel modello relazionale — cardinalità diverse da 1 e N, vincoli di partecipazione per relazioni di grado > 2 ;
- minimizzare i valori NULL.

Le procedure qui presentate sono quelle implementate nella maggior parte dei tool commerciali.

6.2 Da Entity Type a Tabelle

Per ogni entity type E dello schema ER:

1. creare una tabella R con **tutti** gli attributi di E ;
2. porre uno degli attributi chiave di E come chiave primaria di R (se la chiave di E è composta, la chiave di R è l'insieme dei corrispondenti attributi).

Esempio.

```
CREATE TABLE Employees (  
    ssn    CHAR(11),  
    name   CHAR(30),  
    lot    INTEGER,  
    PRIMARY KEY (ssn)  
);
```

6.3 Mapping delle Relazioni (Relationship Types)

6.3.1 Caso Base (senza vincoli)

Data una relazione ER tra due entity types, si crea una nuova tabella con:

- la **chiave primaria di ciascuna entità partecipante** (come chiave esterna);
- gli **attributi descrittivi** della relazione.

Gli attributi non descrittivi di questa nuova tabella formano una **superchiave** (o **chiave candidata**, se non ci sono vincoli di chiave in ER).

Esempio. Employees – Works_In2 (con attributo **since**) – Departments, dove Works_In2 coinvolge anche una terza entità Locations:

```
CREATE TABLE Works_In2 (  
    ssn        CHAR(11),  
    did        INTEGER,  
    address    CHAR(20),  
    since      DATE,  
    PRIMARY KEY (ssn, did, address),  
    FOREIGN KEY (address) REFERENCES Locations,  
    FOREIGN KEY (did) REFERENCES Departments  
);
```

6.3.2 Relazioni Ricorsive

```
CREATE TABLE Reports_To (  
    supervisor_ssn CHAR(11),  
    subordinate_ssn CHAR(11),  
    PRIMARY KEY (supervisor_ssn, subordinate_ssn),  
    FOREIGN KEY (supervisor_ssn) REFERENCES Employees(ssn),  
    FOREIGN KEY (subordinate_ssn) REFERENCES Employees(ssn)  
);
```

6.3.3 Vincolo di Chiave

Se su una delle due entità (es. E_2) esiste un vincolo di chiave, il mapping di base non va bene: la coppia $\langle PK_1, PK_2 \rangle$ sarebbe una superchiave ma **non** una chiave candidata (solo PK_2 non può ripetersi). Due soluzioni:

- A. nuova relazione R_A con PK_1, PK_2 come chiavi esterne, ma **solo** PK_2 come chiave primaria, più eventuali attributi descrittivi;
- B. aggiungere a E_2 un nuovo attributo (chiave esterna) che referencia PK_1 — sfruttando il fatto che PK_2 non può comunque ripetersi, per il vincolo di chiave — più eventuali attributi descrittivi.

Esempio (Employees –Manages→ Departments, vincolo di chiave su Departments).

-- Soluzione A

```
CREATE TABLE Manages (
    ssn    CHAR(11),
    did    INTEGER,
    since  DATE,
    PRIMARY KEY (did),
    FOREIGN KEY (ssn) REFERENCES Employees,
    FOREIGN KEY (did) REFERENCES Departments
);
```

-- Soluzione B

```
CREATE TABLE Dept_Mgr (
    did    INTEGER,
    dname  CHAR(20),
    budget REAL,
    ssn    CHAR(11),
    since  DATE,
    PRIMARY KEY (did),
    FOREIGN KEY (ssn) REFERENCES Employees
);
```

Confronto:

- **B:** *pro* — evita una tabella MANAGES a sé stante, associa direttamente un dipartimento al suo manager senza join; *contro* — molti NULL in **ssn** se ci sono molti dipartimenti senza manager.
- **A:** *pro* — elimina il problema dei NULL; *contro* — molte operazioni richiedono un join tra le due tabelle.

Entrambi gli approcci si estendono a relazioni di grado > 2 .

6.3.4 Vincolo di Partecipazione

- **partecipazione parziale:** non richiede alcuna specifica particolare — corrisponde all'assenza di vincoli;
- **partecipazione totale:** non rappresentabile in modo completo senza perdita di informazione rispetto al modello ER; si può catturare solo combinando specifiche del modello relazionale con strumenti SQL aggiuntivi (*table constraints, assertions* — approfonditi con SQL).

Esempio parziale: aggiungere NOT NULL su `ssn` in `Dept_Mgr` implica che non possa esistere un dipartimento senza manager specificato — cattura **un aspetto** della partecipazione totale di `Departments` in `Manages`, non l'intero vincolo.

Esercizio 1

Nel mapping tra modello ER e modello relazionale, il vincolo di partecipazione parziale non richiede alcuna specifica particolare e corrisponde all'assenza di vincoli. (Falso / È falso, va rappresentato con una linea in grassetto che connette l'entità con partecipazione totale con la relazione / Vero / È vero solo nel caso in cui per l'entità sull'altro lato della relazione viene specificato un vincolo di chiave)

Soluzione: Vero — è esattamente la definizione data sopra: la partecipazione parziale è il caso di default, senza bisogno di alcuna specifica aggiuntiva nel modello relazionale (a differenza della partecipazione totale, che invece richiede accorgimenti extra e comunque non è pienamente rappresentabile).

6.4 Weak Entities

Le entità deboli: partecipano sempre a una relazione **1:N** con l'entità identificante; hanno un **vincolo di chiave**; hanno **partecipazione totale**. Due caratteristiche aggiuntive da gestire nel mapping:

- hanno solo una **chiave parziale**;
- devono essere **cancellate** se viene cancellata l'entità da cui dipendono (ON DELETE CASCADE).

Esempio. Employees –Policy→ Dependents (entità debole, chiave parziale `pname`):

```
CREATE TABLE Dep_Policy (  
    pname CHAR(20),  
    age   INTEGER,  
    cost  REAL,  
    ssn   CHAR(11) NOT NULL,  
    PRIMARY KEY (pname, ssn),  
    FOREIGN KEY (ssn) REFERENCES Employees,  
    ON DELETE CASCADE  
);
```

La chiave primaria combina la chiave parziale dell'entità debole con la chiave dell'entità identificante; NOT NULL rinforza la partecipazione totale (vedi sopra).

Esercizio 2

Data la specifica in ER dell'entità debole Dependents (chiave parziale **pname**, entità identificante Employees con chiave **ssn**, relazione identificante Policy), qual è il suo corretto mapping nel modello relazionale?

- a) `CREATE TABLE Dependents (pname CHAR(20), age INT, cost REAL, PRIMARY KEY (pname), FOREIGN KEY pname REFERENCES Employees (ssn) ON DELETE CASCADE)`
- b) `CREATE TABLE Dependents (pname CHAR(20), age INT, cost REAL, ssn CHAR(11), PRIMARY KEY (ssn, pname), FOREIGN KEY (ssn) REFERENCES Employees(ssn) ON DELETE CASCADE)`
- c) Tutte le opzioni sono valide

Soluzione: b) — coerente con quanto appena visto: la chiave primaria dev'essere (**ssn**, **pname**), non la sola chiave parziale **pname** (opzione a), che da sola non identifica univocamente un Dependent tra impiegati diversi.

6.5 Gerarchie IS-A

Due approcci principali per mappare una gerarchia (es. Employee $\rightarrow$ ISA $\rightarrow$ Hourly_Emps, Contract_Emps):

6.5.1 Metodo Generale

Si crea una tabella per **ogni** classe della gerarchia, incluse le sottoclassi con solo la chiave (ereditata) più gli attributi specifici:

```
CREATE TABLE Employees (ssn CHAR(11), name CHAR(30), lot INTEGER,  
    PRIMARY KEY (ssn));
```

```
CREATE TABLE Hourly_Emps (  
    ssn CHAR(11), hourly_wages REAL, hours_worked INTEGER,  
    PRIMARY KEY (ssn),  
    FOREIGN KEY (ssn) REFERENCES Employees ON DELETE CASCADE  
);
```

```
CREATE TABLE Contract_Emps (  
    ssn CHAR(11), contract_id INTEGER,  
    PRIMARY KEY (ssn),  
    FOREIGN KEY (ssn) REFERENCES Employees ON DELETE CASCADE  
);
```

`ON DELETE CASCADE` assicura che le tuple che referenziano un dipendente vengano cancellate insieme al dipendente stesso.

6.5.2 Metodo di Riduzione

Si eliminano la classe generale e si mappano **solo** le sottoclassi, ciascuna con **tutti** gli attributi ereditati:

```
Hourly_Emps(ssn, name, lot, hourly_wages, hours_worked)
Contract_Emps(ssn, name, lot, contract_id)
```

Richiede che **ogni** impiegato appartenga ad almeno una delle sottoclassi (copertura totale).

6.5.3 Confronto tra i Due Metodi

- **Metodo generale:** si applica sempre; le query sugli attributi di Employees non richiedono di esaminare le altre tabelle; le query su attributi specifici di una sottoclasse possono richiedere un join.
- **Metodo di riduzione:** **non** si applica se esistono impiegati che non appartengono a nessuna sottoclasse; se c'è **overlap**, le informazioni ereditate (name, lot) sono **duplicate** tra le tabelle; le query su **tutti** gli impiegati devono esaminare entrambe le tabelle.

La scelta dipende dalla semantica dei dati e dalla frequenza con cui vengono eseguite specifiche query.

6.6 Aggregazioni

Esempio. Employees –Monitors(until)–> [Projects –Sponsors(since)–> Departments], dove Monitors aggrega Sponsors.

Approccio generale: Employees, Projects, Departments mappate come di consueto; Sponsors mappata come una relazione normale; per Monitors si crea una nuova tabella con le chiavi delle entità coinvolte + gli attributi descrittivi:

```
CREATE TABLE Monitors (
    ssn CHAR(11), did INTEGER, pid INTEGER, until DATE,
    PRIMARY KEY (ssn, did, pid),
    FOREIGN KEY (ssn) REFERENCES Employees,
    FOREIGN KEY (did, pid) REFERENCES Sponsors(did, pid)
);
```

Caso speciale (semplificazione): se **entrambe** le condizioni seguenti valgono:

1. la relazione esterna all'aggregazione (Sponsors) **non ha attributi descrittivi**;
2. l'aggregazione ha **partecipazione totale** in Monitors (ogni coppia progetto-dipartimento deve avere un impiegato che la monitora);

allora **Sponsors può essere rimossa** dal mapping: tutte le sue istanze sono ricavabili dalle colonne $\langle pid, did \rangle$ di Monitors.

7 Dipendenze Funzionali e Chiavi

7.1 Motivazione: Anomalie nella Base di Dati

Una relazione mal progettata può presentare **anomalie**. Esempio: una tabella che unisce impiegati, progetti a cui partecipano (con relativa funzione) e informazioni di stipendio/bilancio, dove per ogni impiegato lo stipendio è unico (indipendente dai progetti) e per ogni progetto il bilancio è unico (indipendente dagli impiegati). Questo causa:

- **Ridondanza:** lo stipendio di un impiegato si ripete in ogni tupla in cui l'impiegato compare;
- **Anomalia di aggiornamento:** se cambia lo stipendio, vanno aggiornate tutte le tuple in cui compare quell'impiegato;
- **Anomalia di inserimento:** non si può inserire un nuovo impiegato senza assegnargli anche un progetto;
- **Anomalia di cancellazione:** se un impiegato smette di partecipare a qualsiasi progetto, va cancellato dalla tabella insieme alle sue informazioni.

Causa: la tabella mescola informazioni eterogenee (impiegati e stipendi; progetti e bilanci; il legame impiegato-progetto-funzione) in un'unica relazione. Per studiare sistematicamente il problema si introduce il concetto di **dipendenza funzionale** (DF).

7.2 Definizione Formale di Dipendenza Funzionale

Sia r una relazione sullo schema $R(X)$, e siano Y, Z due sottoinsiemi non vuoti di X . Z **dipende funzionalmente** da Y ($Y \rightarrow Z$) se, per ogni coppia di tuple $t_1, t_2 \in r$:

$$t_1[Y] = t_2[Y] \implies t_1[Z] = t_2[Z]$$

(cioè: se t_1 e t_2 concordano su Y , allora concordano anche su Z).

7.3 Come si Identificano le DF

Le DF derivano in genere da vincoli del mondo reale sul significato degli attributi, **non** solo dai dati:

- da un'istanza di R si può solo **inferire** che una DF *potrebbe* sussistere (potrebbe non valere su un'altra istanza);
- viceversa, se un'istanza contiene tuple che **violano** una DF, si può concludere con **certezza** che quella DF **non** sussiste.

7.4 DF Banali

Una DF $Y \rightarrow A$ è **banale** se $A \in Y$ (es. Impiegato, Stipendio $\rightarrow$ Stipendio è sempre soddisfatta). Una DF $Y \rightarrow Z$ è **non banale** se nessun attributo di Z appartiene a Y . D'ora in poi si considerano solo DF non banali.

7.5 DF e Chiavi

Data una chiave K di R , esiste sempre una DF tra K e ogni altro attributo di R (per definizione di chiave, non possono esistere due tuple con lo stesso valore su K). Il concetto di DF **generalizza** quindi il vincolo di chiave: una DF $Y \rightarrow Z$ su $R(X)$ **degenera in un vincolo di chiave** se $Y \cup Z = X$ (in tal caso Y è una (super)chiave per $R(X)$).

7.6 DF e Anomalie

Nell'esempio iniziale, $\text{Impiegato} \rightarrow \text{Stipendio}$ e $\text{Progetto} \rightarrow \text{Bilancio}$ **non** contengono una chiave nella parte sinistra (la chiave è $\langle \text{Impiegato}, \text{Progetto} \rangle$), mentre $\text{Impiegato}, \text{Progetto} \rightarrow \text{Funzione}$ sì. **Una DF che contiene una chiave a sinistra non può generare ridondanza** per definizione: le anomalie sono causate esclusivamente dalle DF $Y \rightarrow Z$ in cui Y **non** contiene una chiave.

7.7 Chiusura di un Insieme di Dipendenze Funzionali

Dato un insieme F di DF, è possibile derivare altre DF logicamente implicate da F (es. se $X \rightarrow Y$ e $Y \rightarrow Z$, allora $X \rightarrow Z$). F implica logicamente $X \rightarrow Y$ se ogni istanza che soddisfa F soddisfa anche $X \rightarrow Y$. L'insieme di **tutte** le DF logicamente implicate da F è la **chiusura** di F , indicata F^+ .

7.7.1 Assiomi di Armstrong

F^+ si calcola con gli **Assiomi di Armstrong (AA)**:

- **Riflessività**: se $Y \subseteq X$, allora $X \rightarrow Y$;
- **Aumento**: se $X \rightarrow Y$, allora $XZ \rightarrow YZ$;
- **Transitività**: se $X \rightarrow Y$ e $Y \rightarrow Z$, allora $X \rightarrow Z$.

Regole derivate: **Unione** (se $X \rightarrow Y$ e $X \rightarrow Z$, allora $X \rightarrow YZ$); **Scomposizione** (se $X \rightarrow YZ$, allora $X \rightarrow Y$ e $X \rightarrow Z$).

Gli AA sono **corretti** (*sound*: ogni DF derivata è logicamente implicata da F) e **completi** (*complete*: ogni DF logicamente implicata da F può essere derivata) — generano quindi **tutte e sole** le DF di F^+ .

Esempio. $R(A, B, C, G, H, I)$, $F = \{A \rightarrow B, A \rightarrow C, CG \rightarrow H, CG \rightarrow I, B \rightarrow H\}$. Alcuni membri di F^+ :

- $A \rightarrow H$: per transitività da $A \rightarrow B$ e $B \rightarrow H$;
- $AG \rightarrow I$: aumentando $A \rightarrow C$ con G ($AG \rightarrow CG$), poi transitività con $CG \rightarrow I$;
- $CG \rightarrow HI$: aumentando $CG \rightarrow I$ con CG si ottiene $CG \rightarrow CGI$; aumentando $CG \rightarrow H$ con I si ottiene $CGI \rightarrow HI$; infine transitività tra le due.

7.8 Chiusura di un Insieme di Attributi

Calcolare F^+ per intero è dispendioso (può richiedere un numero esponenziale di DF). Per verificare se $X \rightarrow Y$, basta invece calcolare la **chiusura di un insieme di attributi** X , indicata $(X)^+$:

1. si pone $(X)^+ = X$;
2. se esiste una DF $Z \rightarrow W$ in F con $Z \subseteq (X)^+$, si aggiunge W a $(X)^+$;
3. si ripete finché non è più possibile aggiungere nulla.

Esempio. $F = \{A \rightarrow B, B \rightarrow C, AC \rightarrow D\}$. Chiusura di A : $(A)^+ = A \rightarrow AB$ (usando $A \rightarrow B$) $\rightarrow ABC$ (usando $B \rightarrow C$) $\rightarrow ABCD$ (usando $AC \rightarrow D$). Quindi $(A)^+ = ABCD$.

7.9 Individuazione delle Superchiavi

Dato $\langle R(U), F \rangle$ e $X \subseteq U$: si calcola $(X)^+$; se $(X)^+ = U$ (cioè $X \rightarrow U$), allora X è una **superchiave** di R , altrimenti no. **Attenzione:** X potrebbe essere una superchiave non minimale (non una chiave).

Esempio. $R(A, B, C, D)$, $F = \{A \rightarrow B, B \rightarrow C, AC \rightarrow D\}$: $(A)^+ = ABCD \Rightarrow A$ è superchiave (ed è una chiave, essendo un singolo attributo); $(AC)^+ = ACBD = ABCD \Rightarrow AC$ è superchiave ma **non** è minimale (quindi non è una chiave, basta A); $(B)^+ = BC \Rightarrow B$ **non** è superchiave.

7.9.1 Algoritmo per le Chiavi Candidate

1. **Isolati** (I): attributi né a sinistra né a destra di alcuna DF;
2. **Left** (L): attributi solo a sinistra di qualche DF;
3. **Right** (R): attributi solo a destra di qualche DF;
4. si combinano $I \cup L$;
5. si calcola la chiusura di $I \cup L$: se coincide con U , è l'**unica** chiave candidata e l'algoritmo termina;
6. altrimenti, si individuano gli attributi non ancora considerati (quelli che compaiono sia a sinistra sia a destra, più quelli solo a destra);
7. si combina la chiusura di $I \cup L$ con ciascun sottoinsieme degli attributi dello Step 6, uno alla volta, cercando quali chiusure danno U : sono le chiavi candidate (aggiungere ulteriori attributi produce solo superchiavi non minimali).

Per aiutarsi si può disegnare il **grafo delle dipendenze** (nodi = attributi, archi = DF): un nodo senza archi entranti (isolato incluso) appartiene necessariamente a **ogni** chiave candidata.

Esempio. $R(A, B, C, D, E)$, $F = \{A \rightarrow B, BC \rightarrow E, ED \rightarrow A\}$.

Step 1 (Isolati)	$\{\}$
Step 2 (Left)	$\{C, D\}$
Step 3 (Right)	$\{\}$
Step 4 (unione I, L)	$\{C, D\}$
Step 5 (chiusura)	$(CD)^+ = CD$ (non è chiave)
Step 6 (restanti)	$\{A, B, E\}$
Step 7	$(CDA)^+ = (CDB)^+ = (CDE)^+ = ABCDE$

Aggiungere ulteriori attributi produce solo superchiavi: **ACD**, **BCD** e **CDE** sono le sole chiavi candidate di R (C e D non hanno archi entranti nel grafo delle dipendenze, quindi appartengono a ogni chiave candidata).

Esercizio

Dato $\langle R(A, B, C, D), \{AB \rightarrow C, C \rightarrow D, C \rightarrow A\} \rangle$, elencare tutte le chiavi candidate.

Soluzione: applicando l'algoritmo — Isolati = $\{\}$, Left = $\{B\}$, Right = $\{D\}$; $(B)^+ = B$ (non basta). Restanti = $\{A, C\}$: $(AB)^+$: $AB \rightarrow C$ (ho A,B) dà ABC ; $C \rightarrow D$ dà $ABCD \Rightarrow \mathbf{AB}$ è chiave candidata. $(BC)^+$: $C \rightarrow D$ dà BCD ; $C \rightarrow A$ dà $ABCD \Rightarrow \mathbf{BC}$ è chiave candidata. Chiavi candidate: **AB**, **BC**.

7.10 Copertura Minimale

Una **copertura minimale** F_{min} di F è un insieme di DF logicamente equivalente a F ($F_{min}^+ = F^+$) tale che: (1) ogni DF in F_{min} ha un **solo** attributo a destra; (2) rimuovendo una DF o un attributo dal lato sinistro di una DF in F_{min} , l'insieme non è più equivalente a F . **Nota:** può esistere più di una copertura minimale per F . È dimostrabile che uno schema è in BCNF se e solo se lo è rispetto a F_{min} : per questo ci si può limitare a considerare F_{min} .

Calcolo (3 step):

1. **Normalizzazione:** ogni DF con più attributi a destra ($X \rightarrow ABC \dots$) si scompone in DF con un solo attributo a destra ($X \rightarrow A, X \rightarrow B, X \rightarrow C, \dots$);
2. **Attributi estranei:** per ogni DF $XA \rightarrow B$, si verifica se $B \in (X)^+$ (calcolata SENZA usare A): se sì, A è ridondante e si sostituisce $XA \rightarrow B$ con $X \rightarrow B$;
3. **DF ridondanti:** per ogni DF $X \rightarrow A$, si rimuove temporaneamente da M ottenendo M' ; se $A \in (X)^+$ calcolata su M' , la DF è ridondante e resta rimossa.

Esempio. $R(A, B, C, D, E)$, $F = \{A \rightarrow BCE, CDB \rightarrow A, CD \rightarrow E, E \rightarrow B\}$.

1. Normalizzazione: $A \rightarrow BCE$ diventa $A \rightarrow B, A \rightarrow C, A \rightarrow E$. Insieme: $\{A \rightarrow B, A \rightarrow C, A \rightarrow E, CDB \rightarrow A, CD \rightarrow E, E \rightarrow B\}$;
2. Attributi estranei: $(CD)^+ = CDAEB$ (usando le altre DF) $\Rightarrow B$ è estraneo in $CDB \rightarrow A$, che si riduce a $CD \rightarrow A$. Verificando C e D singolarmente: $(C)^+ = C$, $(D)^+ = D$ — nessuno dei due è ridondante, $CD \rightarrow A$ resta;
3. DF ridondanti: da A si può derivare B senza usare $A \rightarrow B$ (via $A \rightarrow E, E \rightarrow B$) $\Rightarrow A \rightarrow B$ è ridondante; da CD si può derivare E senza usare $CD \rightarrow E$ (via $CD \rightarrow A, A \rightarrow E$) $\Rightarrow CD \rightarrow E$ è ridondante.

$$F_{min} = \{A \rightarrow C, A \rightarrow E, CD \rightarrow A, E \rightarrow B\}$$

8 Normalizzazione

8.1 Normalizzazione via Decomposizione

Le **forme normali** sono condizioni che garantiscono che certe anomalie non possano presentarsi in una relazione. Il processo con cui una relazione che non soddisfa una forma normale viene scomposta in due o più relazioni si chiama **normalizzazione**. L'intuizione: le anomalie si risolvono **decomponendo** la relazione originale in relazioni più piccole. R_1 e R_2 sono una decomposizione di R se:

- R_1 e R_2 contengono ciascuna un sottoinsieme degli attributi di R ;
- $R_1 \cup R_2$ contiene tutti gli attributi di R .

In generale, una decomposizione rappresenta l'informazione di R tramite due **proiezioni** di R .

8.2 Proprietà Desiderabili di una Decomposizione

- **Decomposizione senza perdita di informazione** (*lossless join*): ricomponendo con un join la decomposizione, non si devono ottenere tuple **spurie** (non appartenenti alla relazione iniziale) — questa proprietà **deve sempre** essere garantita;
- **Conservazione delle DF**: la decomposizione deve preservare le DF della relazione originale — questa proprietà, in certi casi, può essere **sacrificata**.

8.2.1 Perdita di Informazione: Esempio

$R(\text{Impiegato}, \text{Progetto}, \text{Sede})$ decomposta in $R_1(\text{Impiegato}, \text{Sede})$ e $R_2(\text{Progetto}, \text{Sede})$: il natural join su **Sede** produce tuple **spurie** (es. un impiegato associato a un progetto a cui non ha mai partecipato, solo perché condividono la sede). **Perché**: **Sede** è l'unico attributo comune, ma non è (e non può essere) chiave di nessuna delle due relazioni — più impiegati e più progetti possono condividere la stessa sede — quindi il join genera tutte le combinazioni possibili, perdendo l'associazione originale impiegato-progetto.

8.2.2 Condizione Formale per la Decomposizione Senza Perdita

Sia R su X , con $X = X_1 \cup X_2$ e $X_0 = X_1 \cap X_2$. R si decompone **senza perdita** su X_1, X_2 se soddisfa $X_0 \rightarrow X_1$ **oppure** $X_0 \rightarrow X_2$. In altre parole: **gli attributi comuni tra le relazioni decomposte devono contenere una (super)chiave di almeno una delle due**. Nell'esempio sopra: **Sede** non contiene una chiave né per R_1 né per $R_2 \Rightarrow$ la decomposizione **non** è senza perdita.

8.2.3 Conservazione delle DF

Condizione: ciascuna DF dello schema originario deve coinvolgere attributi che compaiono tutti insieme in **uno** degli schemi decomposti (le DF originali devono essere ottenibili per proiezione dagli schemi decomposti).

Esempio. Le DF di $R(\text{Impiegato}, \text{Progetto}, \text{Sede})$ sono $\text{Impiegato} \rightarrow \text{Sede}$ e $\text{Progetto} \rightarrow \text{Sede}$. La decomposizione $R_1(\text{Impiegato}, \text{Sede})$, $R_2(\text{Impiegato}, \text{Progetto})$ **non** conserva $\text{Progetto} \rightarrow \text{Sede}$: si potrebbero inserire $\langle \text{Neri}, \text{Milano} \rangle$ in R_1 e $\langle \text{Neri}, \text{Marte} \rangle$ in R_2 senza violare alcuna DF nelle tabelle decomposte, pur violando concettualmente $\text{Progetto} \rightarrow \text{Sede}$ nella relazione originale.

Esercizio 1

Una decomposizione di $R(A, B, C)$ nelle relazioni $S(A, B)$ e $T(B, C)$ è senza perdita di informazione se: (tutte le tuple di R sono nel join di S e T / il join di S e T contiene tutte e sole le tuple che sono nella relazione originale R / se soddisfa la DF $A \rightarrow B$ / la DF $C \rightarrow A$ è soddisfatta)

Soluzione: il join di S e T contiene tutte e sole le tuple che sono nella relazione originale R — è la definizione formale di decomposizione lossless: $R \bowtie S = R$ (né tuple mancanti né tuple spurie).

8.3 Prima Forma Normale (1NF)

Una relazione è in 1NF se **non** ha: tuple ripetute (deve esistere una chiave); attributi composti e/o multi-valore. Molti DBMS non accettano nemmeno relazioni che violano la 1NF.

Esempio. $\text{DEPARTMENT}(\text{Dname}, \text{Dnumber}, \text{Dmgr_ssn}, \text{Dlocations})$ non è in 1NF perché Dlocations ha più valori per la stessa entità (es. Research: {Bellaire, Sugarland, Houston}). Si normalizza ripetendo una riga per ogni località:

Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

8.4 Richiami: Attributi Primi e Non-Primi

- **attributo primo** (*prime*): appartiene ad **almeno una** chiave candidata;
- **attributo non-primo** (*non-prime*): non appartiene ad **alcuna** chiave candidata.

8.5 Seconda Forma Normale (2NF)

R è in 2NF se è in 1NF e **ogni attributo non-primo dipende dall'intera chiave primaria** (nessuna dipendenza parziale da un sottoinsieme proprio della chiave). **Osservazione:** una relazione con chiave primaria **non composta** non può mai violare la 2NF (non può esistere una dipendenza da "parte" di un solo attributo). **Violare la 2NF** causa principalmente **ridondanza** (e le conseguenti anomalie di inserimento/cancellazione/aggiornamento).

8.6 Terza Forma Normale (3NF)

R è in 3NF se è in 2NF e **nessun attributo non-primo dipende transitivamente da una chiave candidata** (non deve valere $X \rightarrow Y, Y \rightarrow Z \Rightarrow X \rightarrow Z$ con X chiave candidata, a meno che Y non sia esso stesso una chiave candidata). **Formulazione equivalente:** per ogni DF non banale $X \rightarrow A$, vale almeno una delle:

- X è una superchiave di R , oppure
- A è un attributo primo di R .

Esempio. (CodImpiegato, Nome, Reparto, TelReparto), chiave CodImpiegato, DF: CodImpiegato $\rightarrow$ Reparto, Reparto $\rightarrow$ TelReparto. Anomalie: il telefono del reparto si ripete per ogni impiegato (ridondanza); un reparto senza impiegati non può avere un telefono registrato. Si normalizza in (CodImpiegato, Nome, Reparto) + (Reparto, TelReparto): entrambe in 3NF, DF conservate, **nessuna perdita** (Reparto, l'attributo comune, determina funzionalmente TelReparto nella seconda tabella).

8.7 3NF e Ridondanza Residua

La 3NF non elimina **tutte** le ridondanze. Esempio: (Prefisso, Numero, Località, Abbonato, Indirizzo) con $\langle$ Prefisso, Numero $\rangle$ chiave e Località $\rightarrow$ Prefisso (dove Prefisso è un attributo **primo**, essendo parte della chiave): la relazione È in 3NF (la condizione "A è primo" è soddisfatta), ma il prefisso si ripete comunque per ogni abbonato della stessa località — **ridondanza**. Per escludere anche $Y \rightarrow Z$ con Y non-primo e Z primo, si definisce una forma normale più restrittiva.

8.8 Forma Normale di Boyce-Codd (BCNF)

R è in BCNF se è in 2NF e, per **ogni** DF non banale $X \rightarrow Y$ su R , X è una **superchiave** di R . Intuitivamente: ogni attributo descrive un'entità "identificata dalla chiave, dall'intera chiave, e da nient'altro che dalla chiave" (W. Kent, 1978). **È più restrittiva della 3NF** ed è teoricamente la forma migliore, ma **non sempre raggiungibile** preservando tutte le DF — in tal caso ci si "accontenta" della 3NF (sempre ottenibile).

8.8.1 Procedura di Decomposizione in BCNF

Data $R(U)$ e una DF $X \rightarrow A$ che viola la BCNF (X non superchiave): decomporre in $R_1(U - A)$ e $R_2(XA)$; ripetere ricorsivamente su R_1/R_2 finché non sono entrambe in BCNF.

Esempio. (Prefisso, Numero, Località, Abbonato, Indirizzo): Località $\rightarrow$ Prefisso viola la BCNF (Località non è chiave). Decomposizione in BCNF: (Numero, Località, Abbonato, Indirizzo) + (Prefisso, Località).

8.8.2 Quando la BCNF Non È Raggiungibile Preservando le DF

Esempio. DF: Dirigente $\rightarrow$ Sede (ogni dirigente opera in una sola sede); Progetto, Sede $\rightarrow$ Dirigente (per ogni sede, un progetto ha un solo dirigente). La relazione **non** è in BCNF

(nella prima DF, Dirigente non è superchiave). **Nessuna decomposizione può preservare entrambe le DF** (la seconda coinvolge tutti gli attributi): non esiste modo di raggiungere la BCNF per questo schema — ci si accontenta della 3NF.

Esercizio 2

INSEGNA(STUDENTE, INSEGNAMENTO, DOCENTE) con
 DF: (STUDENTE, INSEGNAMENTO) $\rightarrow$ DOCENTE,
 DOCENTE $\rightarrow$ INSEGNAMENTO. Decompo-
 sizione: INSEGNA₁(DOCENTE, INSEGNAMENTO),
 INSEGNA₂(DOCENTE, STUDENTE). Quale affermazione è vera? (non
 esiste modo di decomporre INSEGNA in BCNF / la decomposizione non è in BCNF
 / la decomposizione è in BCNF / la decomposizione è in BCNF e preserva tutte le
 DF originali)

Soluzione: la decomposizione è in BCNF (**ma non** preserva tutte le DF) — è esattamente il fenomeno appena visto: in INSEGNA₁ la chiave è DOCENTE (che determina tutto); in INSEGNA₂ la chiave è DOCENTE (idem). Entrambe in BCNF, ma la decomposizione perde (STUDENTE, INSEGNAMENTO) $\rightarrow$ DOCENTE.

Esercizio 3

$\langle R(A, B, C, D, E, F), F = \{AB \rightarrow CD, C \rightarrow EF\} \rangle$, decomposta in $R_1(A, B, C, D)$
 e $R_2(C, E, F)$. Quale affermazione è vera? (in BCNF e conserva tutte le DF / in
 3NF ma non in BCNF / in BCNF ma non conserva le DF)

Soluzione: in BCNF e conserva tutte le DF — in R_1 la chiave è AB (che con $AB \rightarrow CD$ determina tutto lo schema R_1); in R_2 la chiave è C (che con $C \rightarrow EF$ determina tutto R_2). Entrambe in BCNF, e le DF di partenza sono interamente coperte dai due nuovi schemi.

Esercizio 4

$R(A, B, C, D)$ è in BCNF. Sapendo che su R sussiste la DF $D \rightarrow B$, è vero che
 anche $D \rightarrow C$ sussiste? (No / Solo se anche $B \rightarrow D$ sussiste / Sì / Dipende, non è
 possibile saperlo con certezza)

Soluzione: Sì — poiché R è in BCNF, ogni DF non banale come $D \rightarrow B$ implica che il determinante D sia una **superchiave**. Se D è superchiave, determina **tutti** gli altri attributi di R , incluso C .

8.9 Verificare una DF su un'Istanza

Data un'istanza r di R , è **possibile** stabilire con certezza che r **viola** una DF $A \rightarrow B$ (basta trovare due tuple con stesso A e diverso B); non è invece possibile dimostrare che una DF **vale**, dato che una violazione potrebbe comparire in un'istanza futura.

Esercizio 5

Data un'istanza di una relazione S , è possibile determinare se S **non** soddisfa una DF analizzando le tuple presenti nell'istanza. (Vero / Falso)

Soluzione: Vero — se si trovano due tuple con gli stessi valori per gli attributi determinanti ma valori diversi per gli attributi determinati, si ha la prova che la DF è violata.

Esercizio 6

Sia data $R(A, B, C, D)$. La seguente query verifica se la DF $A \rightarrow B$ è soddisfatta da una qualunque istanza di R (0 righe = nessuna violazione; altrimenti una o più tuple violano la DF)?

```
SELECT COUNT(*)  
FROM R AS R1, R AS R2  
WHERE (R1.B != R2.B) AND (R1.A = R2.A)
```

(Vero / Falso)

Soluzione: Vero — la query cerca esattamente le violazioni della definizione di DF: coppie di tuple con lo stesso valore di A ma valori diversi di B .

8.10 Esempio Completo: dalle Chiavi alla BCNF

(Ramakrishnan & Gehrke, Exercise 19.2 — stesso schema usato per l'algoritmo delle chiavi candidate nella sezione precedente.)

Esercizio 7

$R(A, B, C, D, E)$ con $F = \{A \rightarrow B, BC \rightarrow E, ED \rightarrow A\}$. (1) Elencare tutte le chiavi di R . (2) R è in 3NF? (3) R è in BCNF?

Soluzione: (1) Chiavi candidate: **ACD, BCD, CDE** (già calcolate con l'algoritmo delle chiavi candidate — Sez. precedente). Dato che la loro unione copre $\{A, B, C, D, E\}$, **ogni** attributo di R è primo — non esistono attributi non-primi. (2) **È in 3NF**: per ogni DF non banale $X \rightarrow A$ basta che A sia primo, condizione qui sempre soddisfatta. (3) **Non è in BCNF**: la DF $A \rightarrow B$ viola la condizione, perché $(A)^+ = AB \neq ABCDE$ — A non è una superchiave.